

▶ Chapter 16: 좀비 서바이버 생명과 좀비 AI

레트로의 유니티 게임프로그래밍 에센스 (개정판)



이제만 저출

Contents

- Chapter 16 좀비 서바이버 생명과 좀비 AI
 - 16.1 다형성
 - 16.2 LivingEntity 기반 클래스
 - 16.3 플레이어 체력 UI
 - 16.4 PlayerHealth 스크립트
 - 16.5 내비게이션 시스템과 좀비 준비
 - 16.6 Zombie 스크립트
 - 16.7 마치며



CHAPTER 16 좀비 서바이버 생명과 좀비 AI

- 다형성을 사용해 여러 타입을 하나의 타입으로 다루기
- 오버라이드를 사용해 부모 클래스의 기존 메서드 확장하기
- 이벤트를 사용해 견고한 커플링을 해소하고 코드를 간결하게 만들기
- UI 슬라이더 사용하기
- 게임 월드 내부에 UI 배치하기
- 내비게이션 시스템을 사용해 인공지능 구현하기

16.1 다형성(1)

16.1.1 상속 관계에서 다형성

- 상속이란 부모 클래스를 기반으로 자식 클래스를 만드는 방법
 - 자식 클래스는 부모 클래스의 필드와 메서드를 가지고 있음
 - 자식 클래스든 부모 클래스 타입으로 취급하고 사용할 수 있음
- 몬스터 클래스를 만들고 그것을 상속한 오크 클래스와 드래곤 클래스를 만든다고 가정
 - Monster 클래스는 공격력을 나타내는 damage 변수와 공격을 실행하는 Attack() 메서드를 가짐
 - Orc 클래스와 Dragon 클래스는 이러한 Monster 클래스를 상속해서 만들어짐

```
public class Monster : MonoBehaviour {  
    public float damage = 100;  
  
    public void Attack() {  
        Debug.Log("공격!");  
        // 공격 처리...  
    }  
}  
  
public class Orc : Monster {  
    public void WarCry() {  
        Debug.Log("전투함성!");  
        // 전투함성 처리...
```

```
        // 전투함성 처리...  
    }  
}  
  
public class Dragon : Monster {  
    public void Fly() {  
        Debug.Log("날기");  
        // 공중을 나는 처리...  
    }  
}
```

16.1 다형성(2)

- Orc와 Dragon은 Monster의 필드와 메서드는 물론 자신의 고유한 필드와 메서드도 함께 가짐
 - Orc의 WarCry() 메서드는 주변 동료 몬스터의 공격력을 증가시키는 전투함성 스킬
 - Dragon의 Fly() 메서드는 드래곤이 공중을 날아다니는 스킬
 - Orc와 Dragon은 Monster를 상속하므로 Monster 타입으로 취급 가능

```
Orc orc = FindObjectOfType<Orc>(); // 씬에서 오크를 찾음
Monster monster = orc; // 몬스터 타입의 변수에 오크를 할당
monster.Attack(); // 실행 가능
monster.WarCry(); // 실행 불가능(에러)
```

16.1 다형성(3)

16.1.2 다형성을 사용한 패턴

- 다형성을 사용하면 다양한 자식 타입을 하나의 부모 타입으로 다뤄 코드를 쉽고 간결하게 만들수 있음
- Orc의 WarCry() 메서드는 다음과 같은 방식으로 구현

```
public void WarCry() {  
    Debug.Log("전투함성!");  
  
    // 모든 Monster 오브젝트를 찾아 공격력을 10 증가시킴  
    Monster[] monsters = FindObjectsOfType<Monster>();  
    for (int i = 0; i < monsters.Length; i++) {  
        monsters[i].damage += 10;  
    }  
}
```

16.1 다형성(4)

16.1.3 오버라이드

- 메서드에도 다형성을 적용하여 같은 이름의 메서드가 서로 다른 방식으로 동작하게 할 수 있음
- 오버라이드는 부모 클래스에서 작성한 메서드를 자식 클래스에서 재정의하는 것
- 예시) Attack() 메서드가 자식 타입에 따라 서로 다른 공격 대사를 출력

```
public class Monster : MonoBehaviour {  
    public virtual void Attack() {  
        Debug.Log("공격!");  
        // 공격 처리...  
    }  
}  
  
public class Orc : Monster {  
    public override void Attack() {  
        base.Attack();  
        Debug.Log("우리는 노예가 되지 않는다!");  
    }  
}
```

```
    }  
}  
  
public class Dragon : Monster {  
    public override void Attack() {  
        base.Attack();  
        Debug.Log("모든 것이 불타오를 것이다!");  
    }  
}
```

16.1 다형성(5)

- 가상 메서드는 자식 클래스가 오버라이드할 수 있도록 허용된 메서드
- 자식 클래스는 override 키워드를 사용해 부모 클래스의 가상 메서드를 재정의
- 예시) Orc 타입의 오브젝트를 Monster 타입의 변수에 저장하고 Attack() 메서드를 실행

```
Orc orc = FindObjectOfType<Orc>();  
Monster monster = orc;  
monster.Attack(); // Orc의 Attack()이 실행됨
```

16.1 다형성(6)

base

- 자식이 부모의 메서드를 오버라이드할 때는 부모 메서드의 원형을 유지하면서 확장할 수도 있고 완전히 처음부터 메서드를 다시 만들 수도 있음
 - `base.Attack()`;은 부모 클래스인 `Monster`의 `Attack()` 메서드를 실행
 - `base` 키워드는 부모 클래스를 지칭하며, `base`를 사용해 오버라이드되기 전의 원형 메서드로 접근
- 예시의 `Orc`와 `Dragon`은 `Monster`의 `Attack()` 메서드 구현을 유지한 채 자신만의 대사를 추가

```
public class Orc : Monster {  
    public override void Attack() {  
        base.Attack(); // Monster의 Attack()이 실행됨  
        Debug.Log("우리는 노예가 되지 않는다!");  
    }  
}
```

16.1 다형성(7)

오버라이드 활용

- 다양한 자식 클래스 타입이 같은 이름의 메서드를 실행하되 실제 처리는 각자 다르게 만들기
- 예시) Attack() 메서드로 여러 타입의 몬스터가 동시에 공격을 실행하되 각자 공격 대사는 다르게 출력
 - Attack() 메서드는 Monster 타입으로 실행되지만 실제로는 Orc와 Dragon 등 자식 클래스에서 재정의한 Attack() 메서드가 실행

```
Monster[] monsters = FindObjectsOfType<Monster>();  
for (int i = 0; i < monsters.Length; i++) {  
    monsters[i].Attack();  
}
```

16.2 LivingEntity 기반 클래스(1)

- LivingEntity 클래스는 앞의 기능을 적 AI와 플레이어 캐릭터에 생명체로서 갖춰야 하는 기반으로 제공
- LivingEntity를 상속하는 자식 클래스는 LivingEntity의 기존 메서드를 오버라이드하여 자신만의 기능 추가 가능
- 적 AI와 플레이어 캐릭터를 포함해 게임 속 생명체들의 공통 기능
 - 체력을 가진다.
 - 체력을 회복할 수 있다.
 - 공격을 받을 수 있다.
 - 살거나 죽을 수 있다.

16.2 LivingEntity 기반 클래스(2)

16.2.1 LivingEntity 전체 스크립트

```
using System;
using UnityEngine;

// 생명체로 동작할 게임 오브젝트들을 위한 뼈대를 제공
// 체력, 대미지 받아들이기, 사망 기능, 사망 이벤트를 제공
public class LivingEntity : MonoBehaviour, IDamageable {
    public float startingHealth = 100f; // 시작 체력
    public float health { get; protected set; } // 현재 체력
    public bool dead { get; protected set; } // 사망 상태
    public event Action onDeath; // 사망 시 발동할 이벤트

    // 생명체가 활성화될 때 상태를 리셋
    protected virtual void OnEnable() {
        // 사망하지 않은 상태로 시작
        dead = false;
        // 체력을 시작 체력으로 초기화
        health = startingHealth;
    }
}
```

◀ LivingEntity 클래스는 IDamageable을 상속하므로 OnDamage() 메서드를 반드시 구현

▶ 다음 쪽에 이어짐

16.2 LivingEntity 기반 클래스(3)

◀ 앞쪽에 이어

```
// 대미지를 입는 기능
public virtual void OnDamage(float damage, Vector3 hitPoint, Vector3 hitNormal) {
    // 대미지만큼 체력 감소
    health -= damage;

    // 체력이 0 이하 && 아직 죽지 않았다면 사망 처리 실행
    if (health <= 0 && !dead)
    {
        Die();
    }
}

// 체력을 회복하는 기능
public virtual void RestoreHealth(float newHealth) {
    if (dead)
    {
        // 이미 사망한 경우 체력을 회복할 수 없음
        return;
    }
}
```

▶ 다음 쪽에 이어짐

16.2 LivingEntity 기반 클래스(4)

◀ 앞쪽에 이어

```
// 체력 추가
health += newHealth;
}

// 사망 처리
public virtual void Die() {
    // onDeath 이벤트에 등록된 메서드가 있다면 실행
    if (onDeath != null)
    {
        onDeath();
    }

    // 사망 상태를 참으로 변경
    dead = true;
}
}
```

16.2 LivingEntity 기반 클래스(5)

16.2.2 LivingEntity의 필드

- startingHealth - LivingEntity가 활성화될 때 health에 할당될 기본 체력
- health - 현재 체력
- dead - 사망 상태
- health와 dead는 public get, protected set으로 설정된 프로퍼티
 - protected 접근 한정자로 지정된 멤버는 클래스 외부에서는 접근 불가능하지만 자식 클래스에서는 접근 가능
 - 따라서 health와 dead의 값을 클래스 외부에서는 변경할 수 없지만 LivingEntity를 상속하는 자식 클래스에서는 변경 가능
- OnDeath - 사망 시 발동될 이벤트
 - OnDeath 이벤트에는 사망 시 실행할 메서드들이 등록

```
public float startingHealth = 100f;  
public float health { get; protected set; }  
public bool dead { get; protected set; }  
public event Action onDeath;
```

16.2 LivingEntity 기반 클래스(6)

16.2.3 Action

- Action 타입은 입력과 출력이 없는 메서드를 가리킬 수 있는 델리게이트
 - 델리게이트는 '대리자'로 번역되며 메서드를 값으로 할당받을 수 있는 타입
- 마우스를 클릭할 때마다 onClean에 등록된 방청소 메서드가 실행되는 예시
 - Start() 메서드는 onClean에 방을 청소하는 메서드를 등록

```
void Start() {  
    onClean += CleaningRoomA;  
    onClean += CleaningRoomB;  
}
```

- CleaningRoomA()를 먼저 실행하고 그 결과값을 onClean에 더하는 형태가 되므로 에러가 발생

```
// CleanRoomA();의 실행 결과값을 onClean에 추가(에러)  
onClean += CleaningRoomA();
```

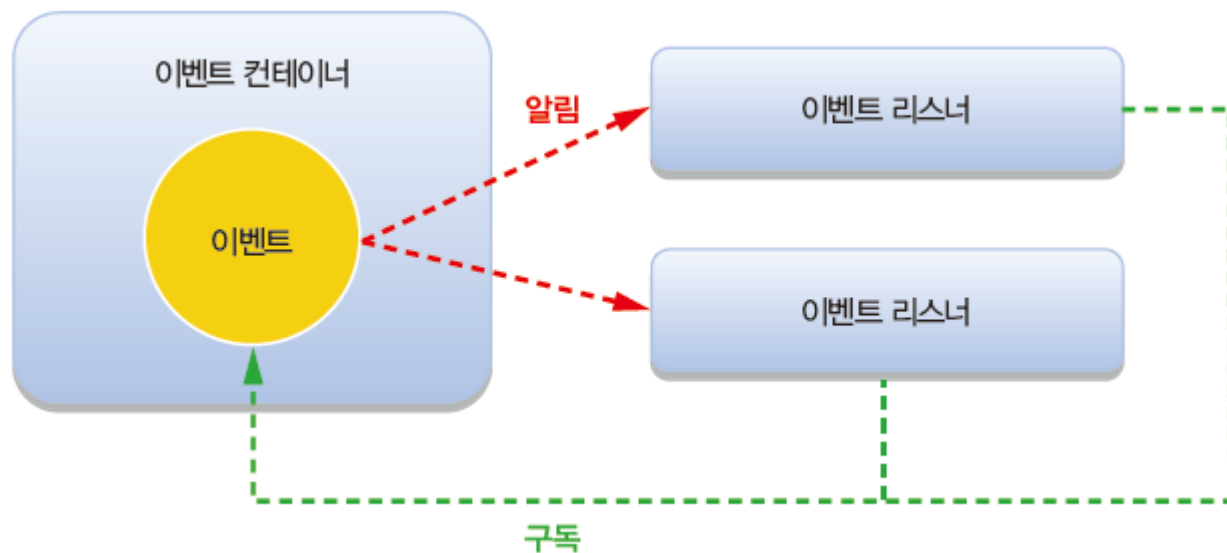
- onClean에 메서드를 등록한 후 onClean();을 실행하면 등록된 메서드가 일괄 실행

```
void Update() {  
    if (Input.GetMouseButtonDown(0)) {  
        onClean(); // CleaningRoomA()와 CleaningRoomB() 실행  
    }  
}
```

16.2 LivingEntity 기반 클래스(7)

16.2.4 이벤트

- 이벤트는 연쇄 동작을 이끌어내는 사건
- 이벤트 자체는 어떤 일을 실행하지 않지만 이벤트가 발생하면 이벤트를 구독하는 처리들이 연쇄적으로 실행
- C#에서 이벤트를 구현하는 대표적인 방법은 델리게이트를 클래스 외부로 공개
 - 외부로 공개된 델리게이트는 클래스 외부의 메서드가 등록될 수 있는 명단이자 이벤트
- 이벤트 리스너 - 이벤트를 항상 듣고 있다가 이벤트가 발동될 때 실행되는 메서드
- '이벤트 구독' - 이벤트 리스너를 이벤트에 등록하는 것



16.2 LivingEntity 기반 클래스(8)

견고한 커플링 해소

- 이벤트는 자신을 구독하는 메서드의 구현과 상관없이 동작하므로 '견고한 커플링' 문제를 해소
- 플레이어가 죽었을 때 게임 데이터를 저장하는 기능을 구현한다고 가정
 - GameData의 구현이 변경되면(예를 들어 Save () 메서드의 이름이 변경되는 경우) Player 클래스의 구현도 변경해야 하므로 코드를 유연하게 유지보수하기 어려움

```
public class Player : MonoBehaviour {  
    public GameData gameData;  
  
    public void Die() {  
        // 실제 사망 처리...  
        gameData.Save();  
    }  
}  
  
public class GameData : MonoBehaviour {  
    public void Save() {  
        Debug.Log("게임 저장...");  
    }  
}
```

16.2 LivingEntity 기반 클래스(9)

- Player 클래스에서 onDeath라는 이벤트를 제공하는 방식으로 개선
 - 변경된 코드에서 Player 클래스는 GameData 타입의 오브젝트를 비롯해 자신의 사망 사건에 관심 있는 상대방 오브젝트를 파악할 필요가 없음
 - Player의 사망 사건에 관심 있는 오브젝트는 Player의 onDeath 이벤트를 구독하면 됨

```
public class Player : MonoBehaviour {  
    public Action onDeath;  
  
    public void Die() {  
        // 실제 사망 처리...  
        onDeath();  
    }  
}
```

```
public class GameData : MonoBehaviour {  
    void Start() {  
        Player player = FindObjectOfType<Player>();  
        player.onDeath += Save;  
    }  
  
    public void Save() {  
        Debug.Log("게임 저장...");  
    }  
}
```

16.2 LivingEntity 기반 클래스(10)

event

- 델리게이트 타입의 변수는 event 키워드를 붙여 선언
 - 어떤 델리게이트 변수를 event로 선언하면 클래스 외부에서는 해당 델리게이트를 실행할 수 없게 됨
- event를 사용하면 이벤트를 소유하지 않은 측에서 멋대로 이벤트를 발동하는 것을 방지

```
public class Player : MonoBehaviour {  
    public event Action onDeath;  
  
    public void Die() {  
        // 실제 사망 처리...  
        onDeath();  
    }  
}
```

```
public class GameData : MonoBehaviour {  
    void Start() {  
        Player player = FindObjectOfType<Player>();  
        player.onDeath += Save;  
        player.onDeath(); // 에러(Player 밖에서는 onDeath 발동 불가)  
    }  
  
    public void Save() {  
        Debug.Log("게임 저장...");  
    }  
}
```

16.2 LivingEntity 기반 클래스(11)

16.2.5 OnEnable()

- LivingEntity의 OnEnable() 메서드는 생명체의 상태를 리셋
- 사망 상태를 false로, 체력을 시작 체력값으로 초기화
- OnEnable() 메서드는 LivingEntity가 활성화될 때 실행

```
protected virtual void OnEnable() {  
    // 사망하지 않은 상태로 시작  
    dead = false;  
    // 체력을 시작 체력으로 초기화  
    health = startingHealth;  
}
```

16.2 LivingEntity 기반 클래스(12)

16.2.6 OnDamage()

- OnDamage() 메서드는 외부에서 LivingEntity를 공격하는 데 사용
 - OnDamage() 메서드는 입력으로 받은 대미지(damage)만큼 현재 체력(health)을 깎음
 - 현재 체력이 0보다 작거나 같고, 아직 사망한 상태가 아니라면 Die() 메서드를 실행해 사망 처리를 실행

```
public virtual void OnDamage(float damage, Vector3 hitPoint, Vector3 hitNormal) {  
    // 대미지만큼 체력 감소  
    health -= damage;  
  
    // 체력이 0 이하 && 아직 죽지 않았다면 사망 처리 실행  
    if (health <= 0 && !dead)  
    {  
        Die();  
    }  
}
```

16.2 LivingEntity 기반 클래스(13)

16.2.7 RestoreHealth()

- RestoreHealth()는 체력을 회복하는 메서드
 - 입력받은 회복량(newHealth)만큼 현재 체력(health)를 증가
 - 이미 죽은 상태에서는 체력을 회복할 수 없으므로 if 문을 사용해 사망 상태에서는 체력을 회복하지 않고 곧장 메서드를 종료

```
public virtual void RestoreHealth(float newHealth) {  
    if (dead)  
    {  
        // 이미 사망한 경우 체력을 회복할 수 없음  
        return;  
    }  
  
    // 체력 추가  
    health += newHealth;  
}
```

16.2 LivingEntity 기반 클래스(14)

16.2.8 Die()

- Die() 메서드는 LivingEntity의 죽음을 구현

```
public virtual void Die() {
```

```
    // onDeath 이벤트에 등록된 메서드가 있다면 실행
```

```
    if (onDeath != null)
```

◀ onDeath 이벤트를 발동하여 이벤트에 등록된 메서드를 실행

```
    {
```

```
        onDeath();
```

```
    }
```

```
    // 사망 상태를 참으로 변경
```

```
    dead = true;
```

◀ dead를 true로 변경해 자신의 상태를 사망한 상태로 변경

```
}
```

16.3 플레이어 체력 UI(1)

- 플레이어 캐릭터의 체력을 구현하기 전에 먼저 체력을 띄울 UI를 구현
- 체력은 원형 슬라이더로 플레이어 캐릭터의 몸체에 표시



▶ 체력 슬라이더의 모습

16.3 플레이어 체력 UI(2)

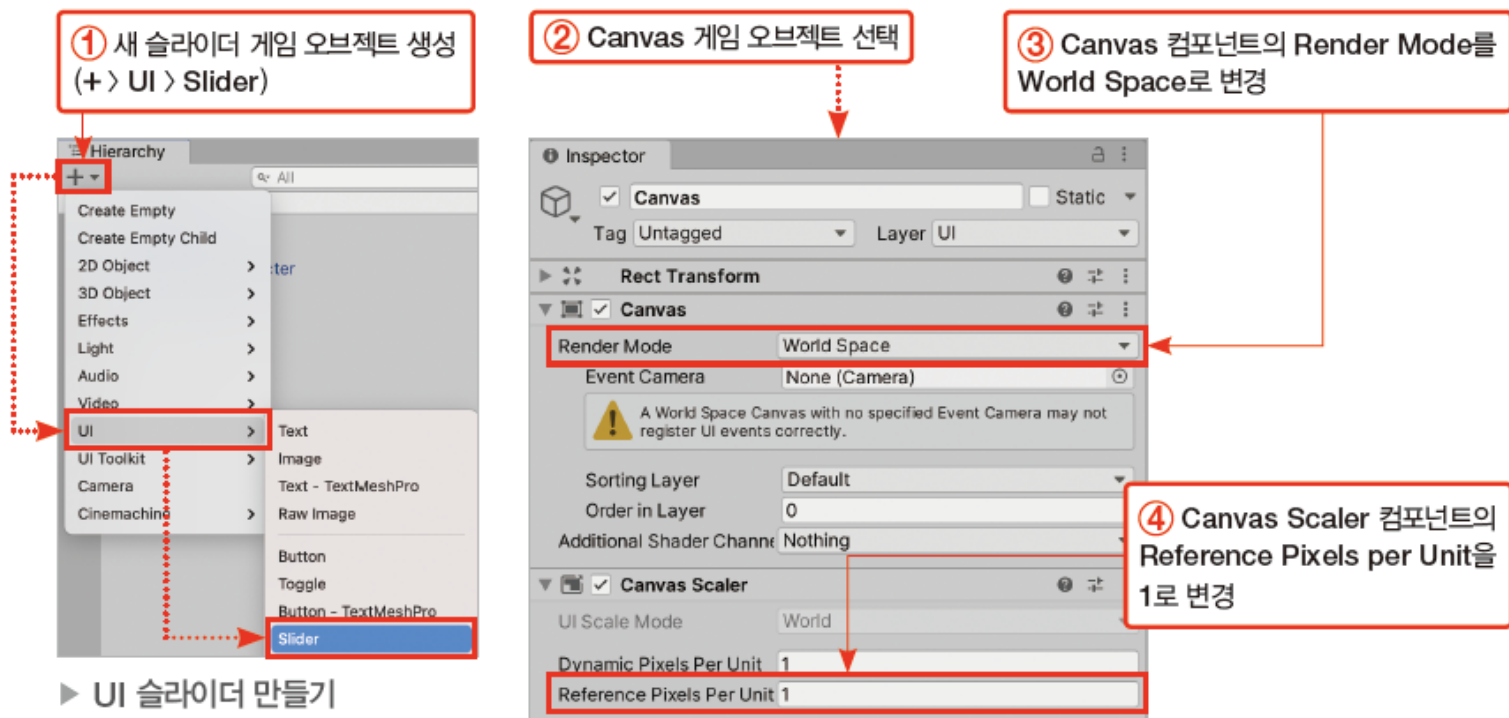
16.3.1 UI 슬라이더 준비

- 체력을 표시할 UI 슬라이더 만들기

[과정 01] UI 슬라이더 만들기

- ① 새 슬라이더 게임 오브젝트 생성(+ > UI > Slider)
- ② 하이어라키 창에서 Canvas 게임 오브젝트 선택
- ③ Canvas 컴포넌트의 Render Mode를 World Space로 변경
- ④ Canvas Scaler 컴포넌트의 Reference Pixels per Unit을 1로 변경

- 3) 체력슬라이더는 3D공간에서 플레이어를 따라다녀야 하므로 캔버스의 랜더모드를 World Space로 변경
- 4) 단위당 레퍼런스 픽셀을 1:1로 하여 UI가 깔끔하게
ex) 100이면 1픽셀이 100유닛에 대응 1 유닛당 0.01 픽셀이 대응



16.3 플레이어 체력 UI(3)

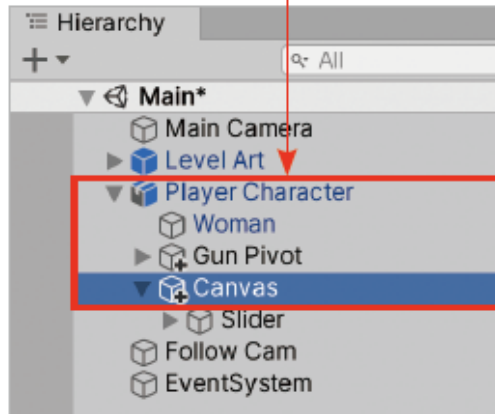
- 캔버스를 플레이어 캐릭터 하체에 배치하고 크기를 변경

90도 회전하여 지면에 맞게 함

[과정 02] 캔버스의 위치와 크기 설정

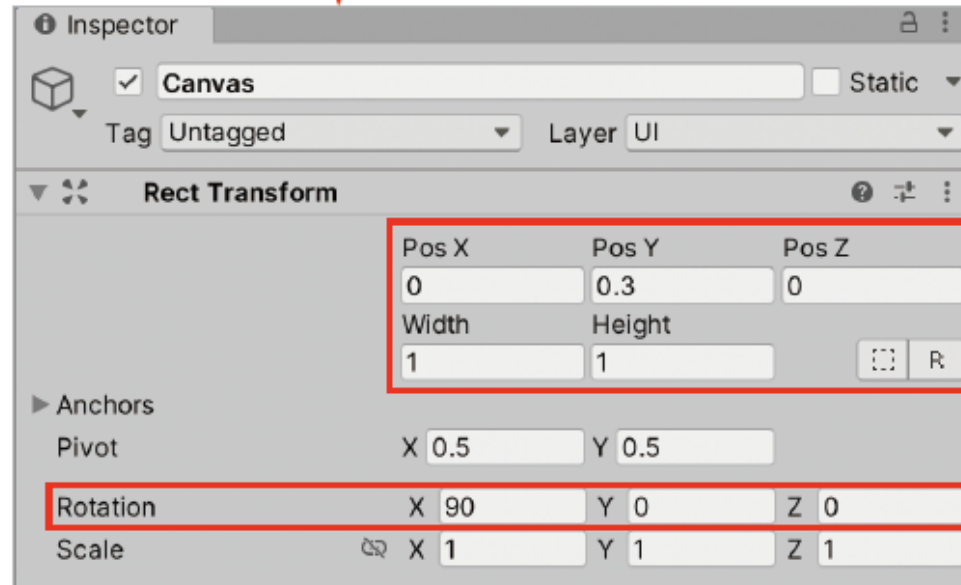
- ① Canvas 게임 오브젝트를 Player Character 게임 오브젝트의 자식으로 만들기(하이어라키 창에서 Canvas를 Player Character로 드래그&드롭)
- ② Canvas 게임 오브젝트 선택
- ③ Rect Transform 컴포넌트의 위치를 (0, 0.3, 0), Width와 Height를 1로 변경
- ④ Rect Transform 컴포넌트의 Rotation을 (90, 0, 0)으로 변경

① Canvas를 Player Character의 자식으로 만들기



▶ 캔버스의 위치와 크기 설정

② Canvas 게임 오브젝트 선택



③ 위치를 (0, 0.3, 0), Width와 Height를 1로 변경

④ Rotation을 (90, 0, 0)으로 변경

16.3 플레이어 체력 UI(4)

- 캔버스의 위치와 크기를 변경하면 캔버스가 아래 그림과 같이 캐릭터 근처에 배치



16.3 플레이어 체력 UI(5)

- 변경된 캔버스 크기에 맞춰 슬라이더 크기를 변경

[과정 03] 슬라이더 크기 변경

- ① 하이어라키 창에서 Canvas 옆의 펼치기 버튼을 [Alt+클릭] → Canvas의 모든 자식 게임 오브젝트가 표시됨
- ② Handle Slide Area 게임 오브젝트를 [Del] 키로 삭제
- ③ Slider, Background, Fill Area, Fill 게임 오브젝트를 모두 선택
- ④ 앵커 프리셋 클릭 > [Alt] 키를 누른 상태에서 우측 하단의 stretch 클릭



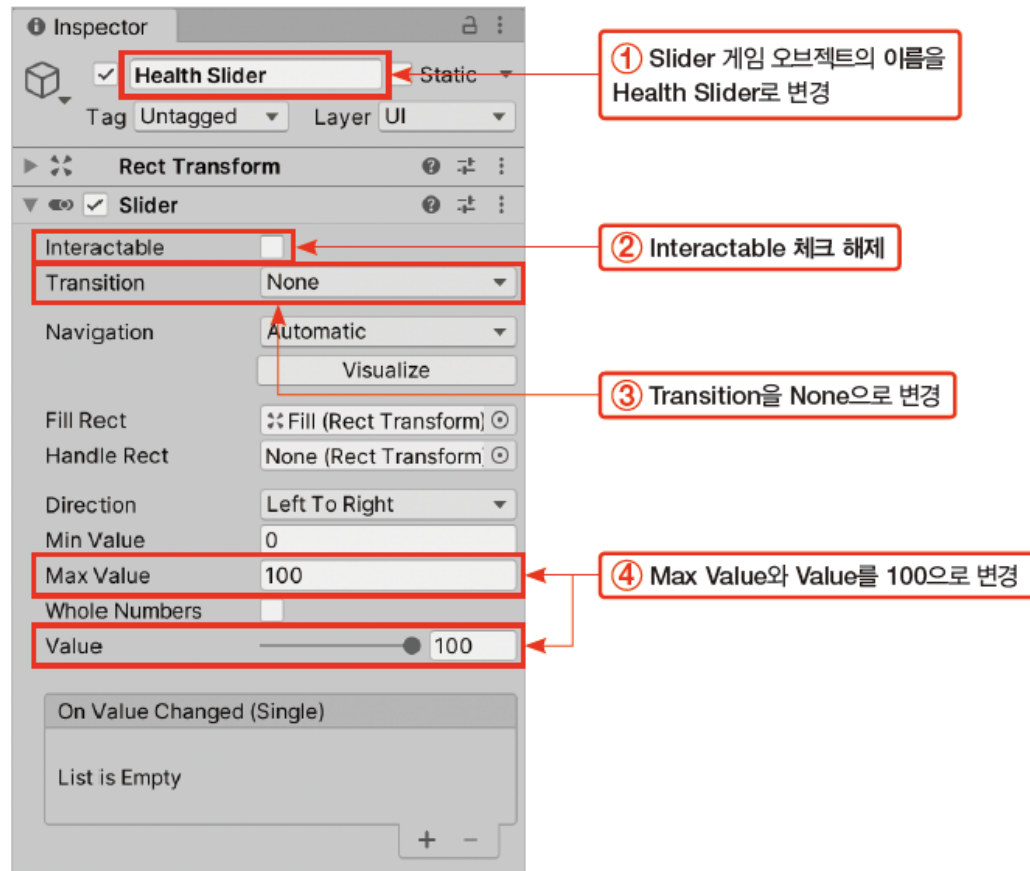
16.3 플레이어 체력 UI(6)

- 슬라이더 컴포넌트의 필드를 설정

[과정 04] 슬라이더 컴포넌트 설정

- ① Slider 게임 오브젝트 선택 > 게임 오브젝트의 이름을 Health Slider로 변경
- ② Slider 컴포넌트에서 Interactable 체크 해제
- ③ Transition을 None으로 변경
- ④ Max Value와 Value를 100으로 변경

Interactable : 상호작용 가능, 체크된 경우 사용자가 클릭이나 드래그 등을 이용하여 UI 게임 오브젝트와 상호작용 가능
Transition : UI와 상호작용시 일어나는 시각 피드백
ex) 마우스 클릭 시 색상 변화



16.3 플레이어 체력 UI(7)

16.3.2 UI 슬라이더의 그래픽 변형

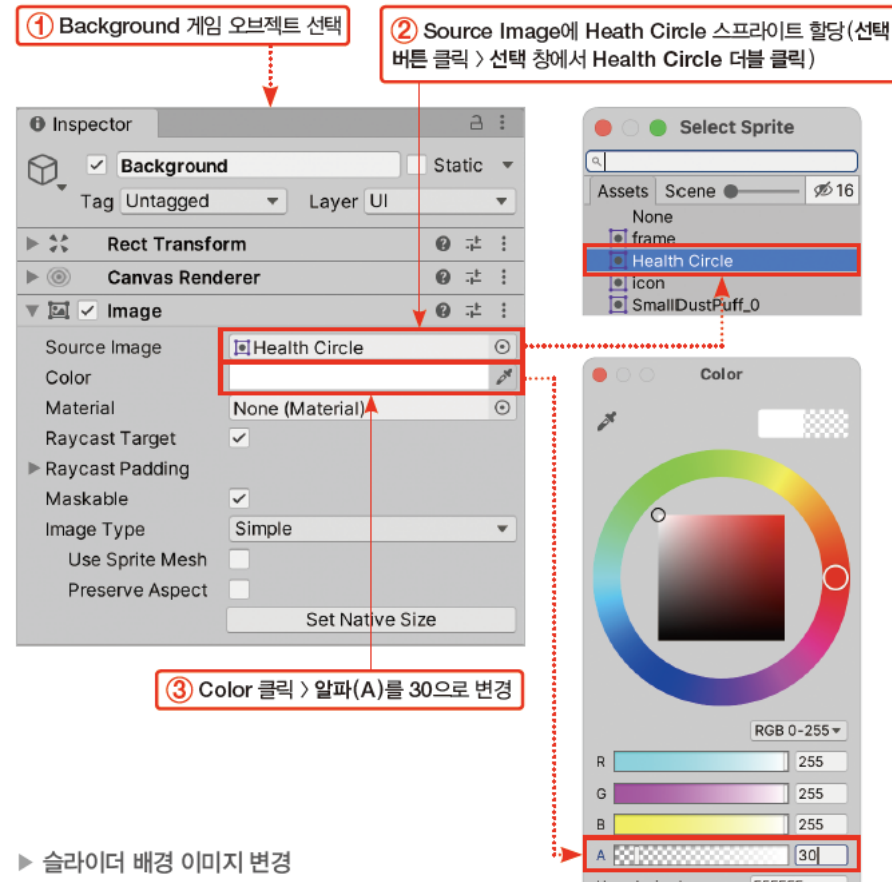
- 슬라이더 컴포넌트는 슬라이더의 배경과 채우기 이미지를 직접 그리지 않음
 - 대신 슬라이더 컴포넌트는 Value 값에 따라 Fill Rect 필드에 할당된 게임 오브젝트의 크기를 조정
 - Fill Rect 필드에 할당된 게임 오브젝트의 크기는 해당 게임 오브젝트의 부모 게임 오브젝트에 상대적으로 결정
- 다음과 같은 상태를 가정
 - 슬라이더 컴포넌트의 최솟값이 0, 최댓값이 100, 현재값이 50
 - 슬라이더 컴포넌트의 Fill Rect에 어떤 UI 게임 오브젝트 A가 할당됨
 - 게임 오브젝트 A는 게임 오브젝트 B의 자식임
- → 슬라이더 컴포넌트는 A 게임 오브젝트의 크기를 , B의 크기의 50%의 크기로 줄임
만약 현재값이 100이라면 슬라이더 컴포넌트는 A 게임 오브젝트의 크기가 B 게임 오브젝트와 100% 일치하도록 늘림

16.3 플레이어 체력 UI(8)

- 슬라이더 배경 이미지 변경

[과정 01] 슬라이더 배경 이미지 변경

- ① 하이어라키 창에서 Background 게임 오브젝트 선택
- ② Image 컴포넌트의 Source Image에 Heath Circle 스프라이트 할당(Source Image 옆의 선택 버튼 클릭 > 선택 창에서 Health Circle 더블 클릭)
- ③ Image 컴포넌트의 Color 필드 클릭 > 알파(A)를 30으로 변경

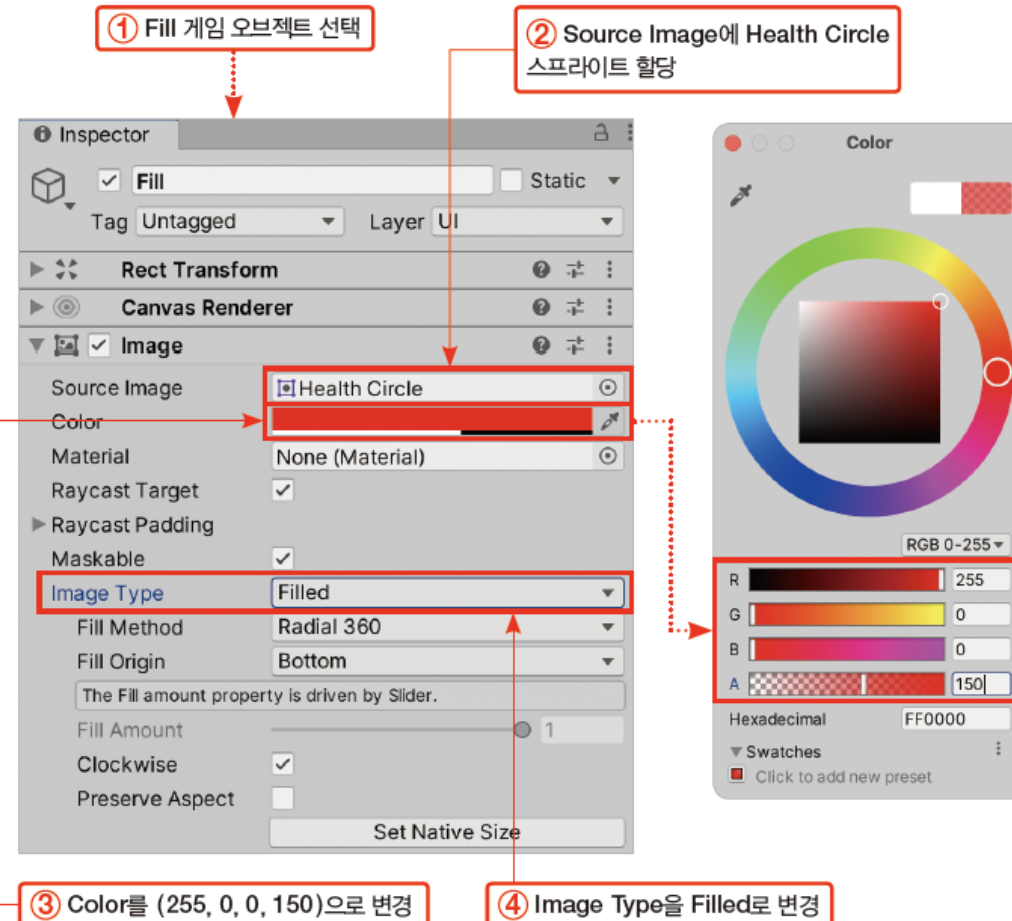


▶ 슬라이더 배경 이미지 변경

16.3 플레이어 체력 UI(9)

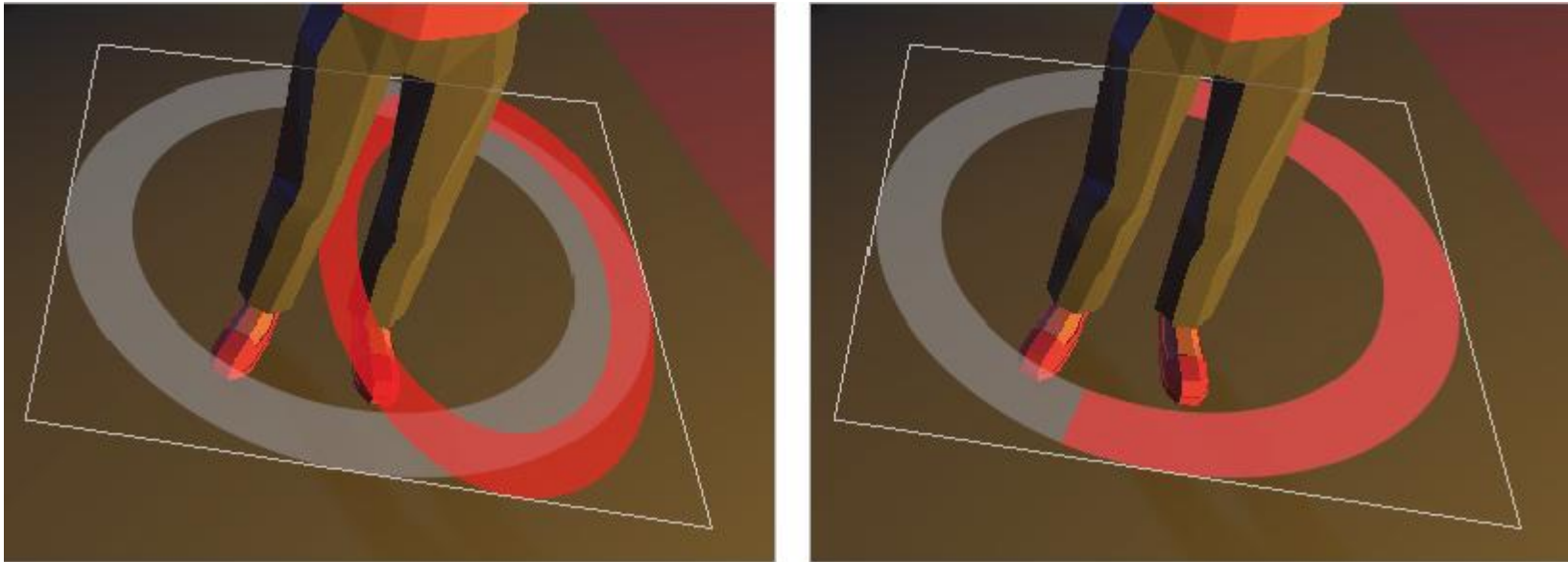
[과정 02] 슬라이더 채우기 이미지 변경

- ① Fill 게임 오브젝트 선택
- ② Image 컴포넌트의 Source Image에 Health Circle 스프라이트 할당
- ③ Image 컴포넌트의 Color 필드 클릭 > 컬러를 (255, 0, 0, 150)으로 변경
- ④ Image Type을 Filled로 변경



16.3 플레이어 체력 UI(10)

- Fill 게임 오브젝트의 이미지 컴포넌트의 이미지 타입(Image Type)이 단순(Simple)인 경우
 - 슬라이더 컴포넌트는 Fill 게임 오브젝트를 단순히 가로나 세로로 잡아 늘려서 슬라이더를 채움
- 우리는 이미지 타입을 채움(Filled)으로 변경
 - Fill 게임 오브젝트가 원형으로 슬라이더를 채움



▶ 단순과 채움의 차이

16.4 PlayerHealth 스크립트(1)

- LivingEntity 클래스를 확장하여 플레이어 캐릭터의 체력을 구현하는 PlayerHealth 스크립트를 구현
- PlayerHealth 스크립트의 기능
 - LivingEntity의 생명체 기본 기능
 - 체력이 변경되면 체력 슬라이더에 반영
 - 공격받으면 피격 효과음 재생
 - 사망 시 플레이어의 다른 컴포넌트를 비활성화
 - 사망 시 사망 효과음과 사망 애니메이션 재생
 - 아이템을 감지하고 사용

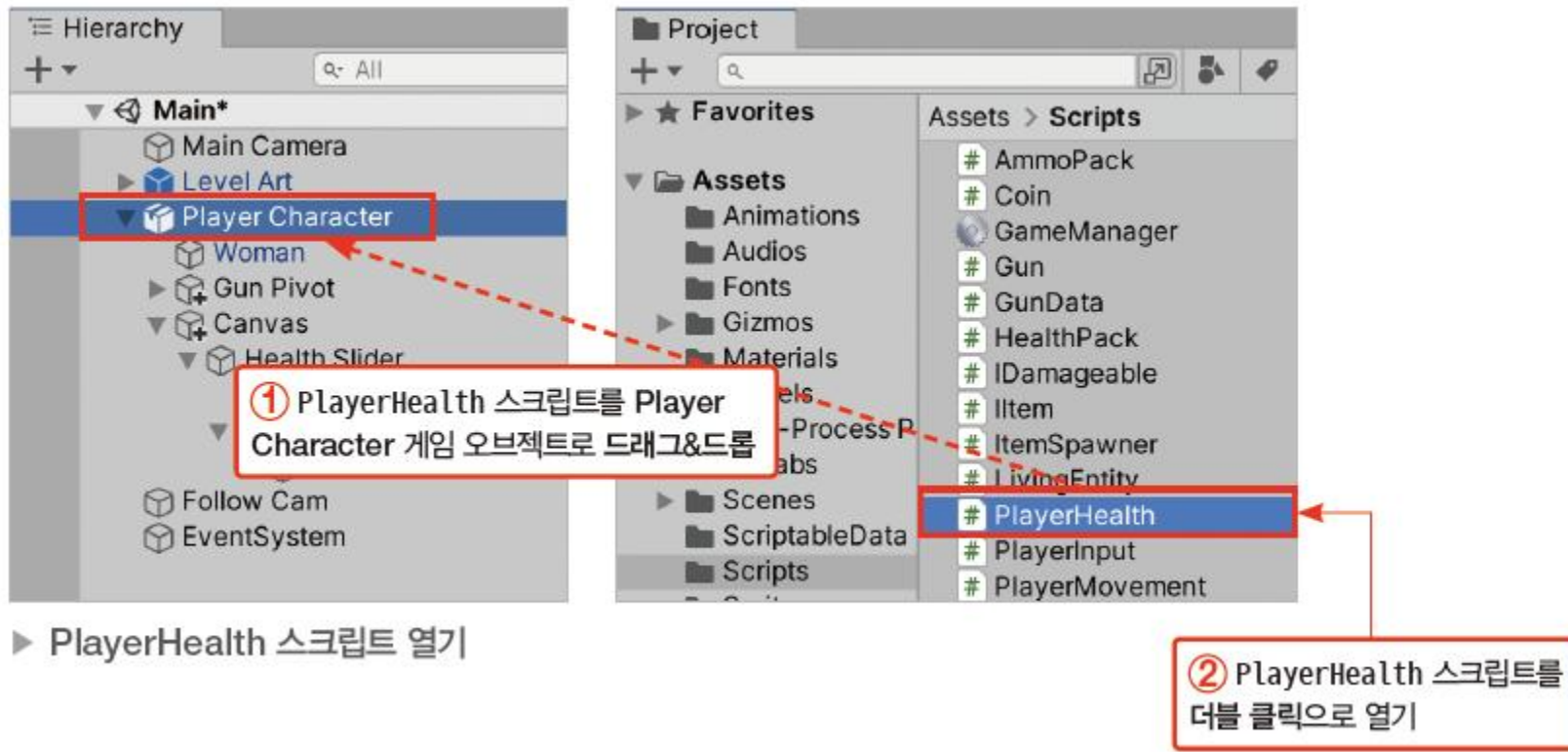
16.4 PlayerHealth 스크립트(2)

16.4.1 PlayerHealth 스크립트 열기

- PlayerHealth 스크립트를 플레이어 캐릭터에 추가하고 스크립트를 열기

[과정 01] PlayerHealth 스크립트 열기

- ① Scripts 폴더의 PlayerHealth 스크립트를 Player Character 게임 오브젝트로 드래그&드롭
- ② PlayerHealth 스크립트를 더블 클릭으로 열기



16.4 PlayerHealth 스크립트(3)

16.4.2 PlayerHealth의 필드

- UI 체력 슬라이더를 할당할 healthSlider가 선언
 - healthSlider에는 16.3절 '플레이어 체력 UI'에서 제작한 Health Slider 게임 오브젝트의 슬라이더 컴포넌트가 할당

```
public Slider healthSlider;
```

- 상황에 따라 재생할 오디오 클립 변수가 선언

```
public AudioClip deathClip;  
public AudioClip hitClip;  
public AudioClip itemPickupClip;
```

- 사용할 컴포넌트에 대한 변수가 선언

```
private AudioSource playerAudioPlayer;  
private Animator playerAnimator;
```

- playerAudioPlayer에는 오디오 소스 컴포넌트, playerAnimator에는 애니메이터 컴포넌트가 할당

```
private PlayerMovement playerMovement;  
private PlayerShooter playerShooter;
```

16.4 PlayerHealth 스크립트(4)

16.4.3 Awake()

- Awake() 메서드는 플레이어 게임 오브젝트에서 필요한 컴포넌트를 찾아 변수에 할당하고 수치를 초기화

[과정 01] Awake () 메서드 완성하기

① Awake() 메서드를 다음과 같이 완성

```
private void Awake() {  
    // 사용할 컴포넌트 가져오기  
    playerAnimator = GetComponent<Animator>();  
    playerAudioPlayer = GetComponent<AudioSource>();  
  
    playerMovement = GetComponent<PlayerMovement>();  
    playerShooter = GetComponent<PlayerShooter>();  
}
```

◀ Player Character 게임 오브젝트에서 애니메이터 컴포넌트와 오디오 소스 컴포넌트를 가져옴

◀ Player Character 게임 오브젝트에서 PlayerMovement와 PlayerShooter 컴포넌트를 가져옴

16.4 PlayerHealth 스크립트(5)

16.4.4 OnEnable()

- OnEnable() 메서드에서는 PlayerHealth 컴포넌트가 활성화될 때마다 체력 상태를 리셋하는 처리를 구현

[과정 01] OnEnable () 메서드 완성하기

① OnEnable() 메서드를 다음과 같이 완성

```
protected override void OnEnable() {  
    // LivingEntity의 OnEnable() 실행(상태 초기화)  
    base.OnEnable();  
  
    // 체력 슬라이더 활성화  
    healthSlider.gameObject.SetActive(true);  
    // 체력 슬라이더의 최댓값을 기본 체력값으로 변경  
    healthSlider.maxValue = startingHealth;  
    // 체력 슬라이더의 값을 현재 체력값으로 변경  
    healthSlider.value = health;  
  
    // 플레이어 조작을 받는 컴포넌트 활성화  
    playerMovement.enabled = true;  
    playerShooter.enabled = true;  
}
```

- ◀ LivingEntity의 OnEnable() 메서드는 health의 값을 startingHealth의 값으로 초기화
- ◀ 체력 슬라이더 게임 오브젝트를 활성화
- ◀ 체력 슬라이더의 최댓값은 시작 체력인 startingHealth로 변경
- ◀ 체력 슬라이더의 현재값은 현재 체력인 health로 변경
- ◀ PlayerShooter와 PlayerMovement 컴포넌트를 활성화

16.4 PlayerHealth 스크립트(6)

16.4.5 RestoreHealth()

- PlayerHealth 클래스의 RestoreHealth() 메서드는 LivingEntity 클래스의 RestoreHealth() 메서드를 오버라이드

[과정 01] RestoreHealth () 메서드 완성하기

① RestoreHealth() 메서드를 다음과 같이 완성

```
public override void RestoreHealth(float newHealth) {  
    // LivingEntity의 RestoreHealth() 실행(체력 증가)  
    base.RestoreHealth(newHealth);  
    // 갱신된 체력으로 체력 슬라이더 갱신  
    healthSlider.value = health;  
}
```

16.4 PlayerHealth 스크립트(7)

16.4.6 OnDamage()

- PlayerHealth 클래스의 OnDamage() 메서드는 LivingEntity 클래스의 OnDamage() 메서드를 오버라이드

[과정 01] OnDamage () 메서드 완성하기

① OnDamage() 메서드를 다음과 같이 완성

```
public override void OnDamage(float damage, Vector3 hitPoint, Vector3 hitDirection) {  
    if (!dead)  
    {  
        // 사망하지 않은 경우에만 효과음 재생  
        playerAudioPlayer.PlayOneShot(hitClip);  
    }  
  
    // LivingEntity의 OnDamage() 실행(데미지 적용)  
    base.OnDamage(damage, hitPoint, hitDirection);  
    // 갱신된 체력을 체력 슬라이더에 반영  
    healthSlider.value = health;  
}
```

16.4 PlayerHealth 스크립트(8)

- 피격 사운드를 재생
 - 단, 플레이어 캐릭터가 이미사 망한 상태에서 공격을 받았을 때 효과음이 재생되는 것을 막기 위해 if 문으로 사망하지 않은 경우에만 효과음을 재생

```
if (!dead)
{
    playerAudioPlayer.PlayOneShot(hitClip);
}
```

- LivingEntity의 OnDamage() 메서드를 실행해 대미지를 적용

```
base.OnDamage(damage, hitPoint, hitDirection);
```

- 대미지가 적용되어 변경된 체력을 체력 슬라이더에 반영

```
healthSlider.value = health;
```

16.4 PlayerHealth 스크립트(9)

16.4.7 Die()

- PlayerHealth 클래스의 Die() 메서드는 LivingEntity 클래스의 Die() 메서드를 오버라이드

[과정 01] Die () 메서드 완성

① Die() 메서드를 다음과 같이 완성

```
public override void Die() {  
    // LivingEntity의 Die() 실행(사망 적용)  
    base.Die();  
  
    // 체력 슬라이더 비활성화  
    healthSlider.gameObject.SetActive(false);  
  
    // 사망음 재생  
    playerAudioPlayer.PlayOneShot(deathClip);  
    // 애니메이터의 Die 트리거를 발동시켜 사망 애니메이션 재생  
    playerAnimator.SetTrigger("Die");  
  
    // 플레이어 조작을 받는 컴포넌트 비활성화  
    playerMovement.enabled = false;  
    playerShooter.enabled = false;  
}
```

16.4 PlayerHealth 스크립트(10)

- base.Die()를 실행하여 LivingEntity의 사망 처리를 실행한 다음, 체력 슬라이더의 게임 오브젝트를 비활성화

```
base.Die();  
healthSlider.gameObject.SetActive(false);
```

- 오디오 소스로 사망 효과음을 재생하고 애니메이터의 Die 트리거를 발동시켜 사망 애니메이션을 재생

```
playerAudioPlayer.PlayOneShot(deathClip);  
playerAnimator.SetTrigger("Die");
```

- Player Character 게임 오브젝트에 추가된 PlayerShooter와 PlayerMovement 컴포넌트를 비활성화

```
playerMovement.enabled = false;  
playerShooter.enabled = false;
```

16.4 PlayerHealth 스크립트(11)

16.4.8 OnTriggerEnter()

- OnTriggerEnter() 메서드는 트리거 충돌한 상대방 게임 오브젝트가 아이템인지 판단하고 아이템을 사용하는 처리를 구현

[과정 01] OnTriggerEnter () 메서드 완성

① OnTriggerEnter() 메서드를 다음과 같이 완성

```
private void OnTriggerEnter(Collider other) {  
    // 아이템과 충돌한 경우 해당 아이템을 사용하는 처리  
    // 사망하지 않은 경우에만 아이템 사용 가능  
    if (!dead)  
    {  
        // 충돌한 상대방으로부터 IItem 컴포넌트 가져오기 시도  
        IItem item = other.GetComponent<IItem>();  
  
        // 충돌한 상대방으로부터 IItem 컴포넌트를 가져오는 데 성공했다면  
        if (item != null)
```

```
{  
    // Use 메서드를 실행하여 아이템 사용  
    item.Use(gameObject);  
    // 아이템 습득 소리 재생  
    playerAudioPlayer.PlayOneShot(itemPickupClip);  
}  
}
```

16.4 PlayerHealth 스크립트(12)

- if (!dead)를 사용해 사망하지 않은 경우에만 아이템을 사용
- if 문에서 GetComponent () 메서드로 충돌한 상대방 게임 오브젝트에서 IItem 타입의 컴포넌트를 가져오기 시도
- IItem 타입의 컴포넌트를 가져오는 데 성공했다면,
 - 해당 IItem 컴포넌트의 Use () 메서드를 실행하여 아이템을 사용
 - 자신의 게임 오브젝트를 Use () 메서드에 아이템 사용 대상으로 입력
 - 오디오 소스의 PlayOneShot () 메서드로 아이템을 줍는 오디오 클립을 재생

```
IItem item = other.GetComponent<IItem>();  
  
if (item != null)  
{  
    item.Use(gameObject);  
    playerAudioPlayer.PlayOneShot(itemPickupClip);  
}
```

16.4 PlayerHealth 스크립트(13)

16.4.9 완성된 PlayerHealth 스크립트

```
using UnityEngine;
using UnityEngine.UI; // UI 관련 코드

// 플레이어 캐릭터의 생명체로서의 동작을 담당
public class PlayerHealth : LivingEntity {
    public Slider healthSlider; // 체력을 표시할 UI 슬라이더

    public AudioClip deathClip; // 사망 소리
    public AudioClip hitClip; // 피격 소리
    public AudioClip itemPickupClip; // 아이템 습득 소리

    private AudioSource playerAudioPlayer; // 플레이어 소리 재생기
    private Animator playerAnimator; // 플레이어의 애니메이터

    private PlayerMovement playerMovement; // 플레이어 움직임 컴포넌트
    private PlayerShooter playerShooter; // 플레이어 슈터 컴포넌트
```

▶ 다음 쪽에 이어짐

16.4 PlayerHealth 스크립트(14)

◀ 앞쪽에 이어

```
private void Awake() {  
    // 사용할 컴포넌트 가져오기  
    playerAnimator = GetComponent<Animator>();  
    playerAudioPlayer = GetComponent<AudioSource>();  
  
    playerMovement = GetComponent<PlayerMovement>();  
    playerShooter = GetComponent<PlayerShooter>();  
}
```

```
protected override void OnEnable() {  
    // LivingEntity의 OnEnable() 실행(상태 초기화)  
    base.OnEnable();  
  
    // 체력 슬라이더 활성화  
    healthSlider.gameObject.SetActive(true);  
    // 체력 슬라이더의 최댓값을 기본 체력값으로 변경  
    healthSlider.maxValue = startingHealth;  
    // 체력 슬라이더의 값을 현재 체력값으로 변경  
    healthSlider.value = health;
```

▶ 다음 쪽에 이어짐

16.4 PlayerHealth 스크립트(15)

◀ 앞쪽에 이어

```
// 플레이어 조작을 받는 컴포넌트 활성화
playerMovement.enabled = true;
playerShooter.enabled = true;
}

// 체력 회복
public override void RestoreHealth(float newHealth) {
    // LivingEntity의 RestoreHealth() 실행(체력 증가)
    base.RestoreHealth(newHealth);

    // 갱신된 체력으로 체력 슬라이더 갱신
    healthSlider.value = health;
}

// 대미지 처리
public override void OnDamage(float damage, Vector3 hitPoint, Vector3 hitDirection) {
    if (!dead)
    {
        // 사망하지 않은 경우에만 효과음 재생
        playerAudioPlayer.PlayOneShot(hitClip);
    }
}
```

▶ 다음 쪽에 이어짐

16.4 PlayerHealth 스크립트(16)

◀ 앞쪽에 이어

```
// LivingEntity의 OnDamage() 실행(대미지 적용)
base.OnDamage(damage, hitPoint, hitDirection);
// 갱신된 체력을 체력 슬라이더에 반영
healthSlider.value = health;
}

// 사망 처리
public override void Die() {
    // LivingEntity의 Die() 실행(사망 적용)
    base.Die();

    // 체력 슬라이더 비활성화
    healthSlider.gameObject.SetActive(false);

    // 사망음 재생
    playerAudioPlayer.PlayOneShot(deathClip);
    // 애니메이터의 Die 트리거를 발동시켜 사망 애니메이션 재생
    playerAnimator.SetTrigger("Die");
}
```

▶ 다음 쪽에 이어짐

16.4 PlayerHealth 스크립트(17)

◀ 앞쪽에 이어

```
// 플레이어 조작을 받는 컴포넌트 비활성화
playerMovement.enabled = false;
playerShooter.enabled = false;
}

private void OnTriggerEnter(Collider other) {
    // 아이템과 충돌한 경우 해당 아이템을 사용하는 처리
    // 사망하지 않은 경우에만 아이템 사용 가능
    if (!dead)
    {
        // 충돌한 상대방으로부터 IItem 컴포넌트 가져오기 시도
        IItem item = other.GetComponent<IItem>();

        // 충돌한 상대방으로부터 IItem 컴포넌트를 가져오는 데 성공했다면
        if (item != null)
        {
```

▶ 다음 쪽에 이어짐

16.4 PlayerHealth 스크립트(18)

◀ 앞쪽에 이어

```
// Use 메서드를 실행하여 아이템 사용
item.Use(gameObject);
// 아이템 습득 소리 재생
playerAudioPlayer.PlayOneShot(itemPickupClip);
    }
}
}
```

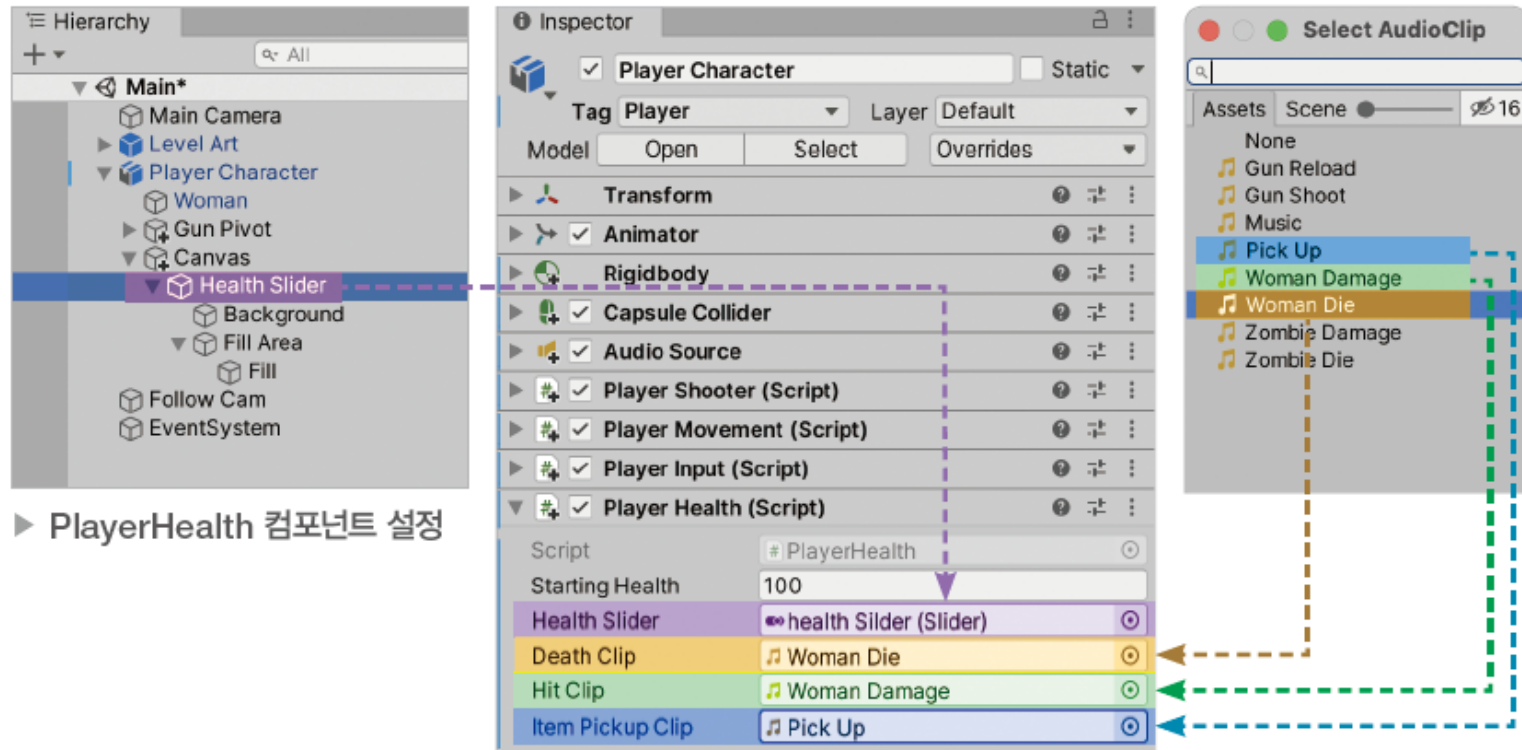
16.4 PlayerHealth 스크립트(19)

16.4.10 PlayerHealth 컴포넌트 설정

- Player Character 게임 오브젝트에 추가된 PlayerHealth 컴포넌트의 필드를 설정

[과정 01] PlayerHealth 컴포넌트 설정

- ① 하이어라키 창에서 Player Character 게임 오브젝트 선택
- ② Player Health 컴포넌트의 Health Slider 필드에 Health Slider 게임 오브젝트 할당
- ③ Death Clip 필드에 Woman Die 오디오 클립 할당
- ④ Hit Clip 필드에 Woman Damage 오디오 클립 할당
- ⑤ Item Pickup Clip 필드에 Pick Up 오디오 클립 할당

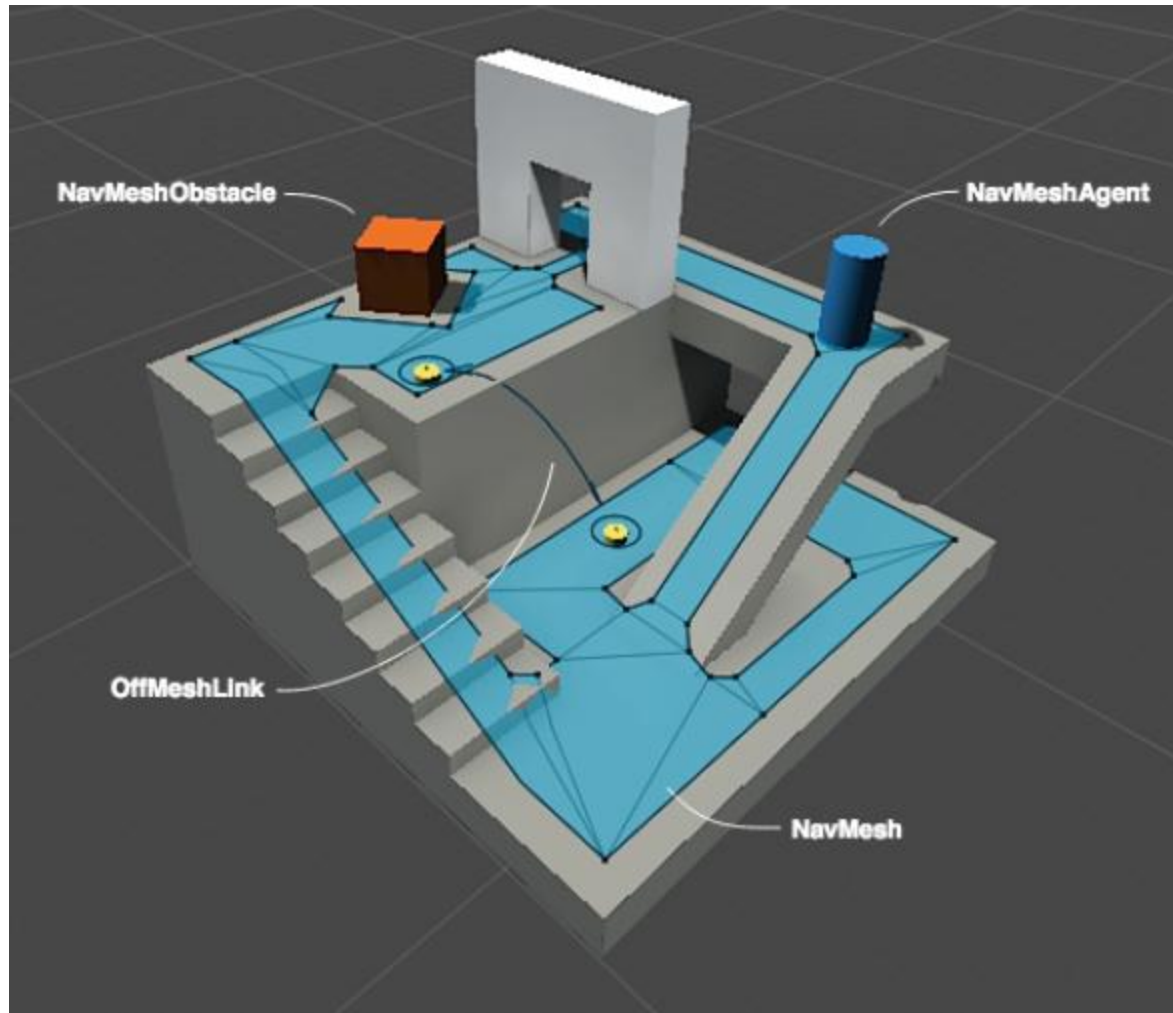


16.5 내비게이션 시스템과 준비 준비(1)

16.5.1 내비게이션 시스템

- 유니티는 한 위치에서 다른 위치로의 경로를 계산하고 실시간으로 장애물을 피하며 이동하는 인공지능을 만드는 내비게이션 시스템을 제공
- 내비게이션 시스템에 사용되는 오브젝트
 - 내비메시(NavMesh) : 에이전트가 걸어 다닐 수 있는 표면
 - 내비메시 에이전트(NavMesh Agent) : 내비메시 위에서 경로를 계산하고 이동하는 캐릭터 또는 컴포넌트
 - 내비메시 장애물(NavMesh Obstacle) : 에이전트의 경로를 막는 장애물
 - 오프메시 링크(Off Mesh Link) : 끊어진 내비메시 영역 사이를 잇는 연결 지점

16.5 내비게이션 시스템과 좀비 준비(2)



▶ 내비게이션 시스템의 구성 요소

16.5 내비게이션 시스템과 좀비 준비(3)

16.5.2 내비메시 빌드

- 내비메시는 게임 월드에서 내비메시 에이전트가 걸어 다닐 표면

[과정 01] 내비메시 굽기

- ① 내비게이션 창 열기(Window > AI > Navigation)
- ② 내비게이션 창에서 Bake 탭 클릭
- ③ Agent Radius를 0.4, Agent Height를 1.8로 변경
- ④ Bake 버튼 클릭

[과정 01] 에이전트 타입 설정 변경하기

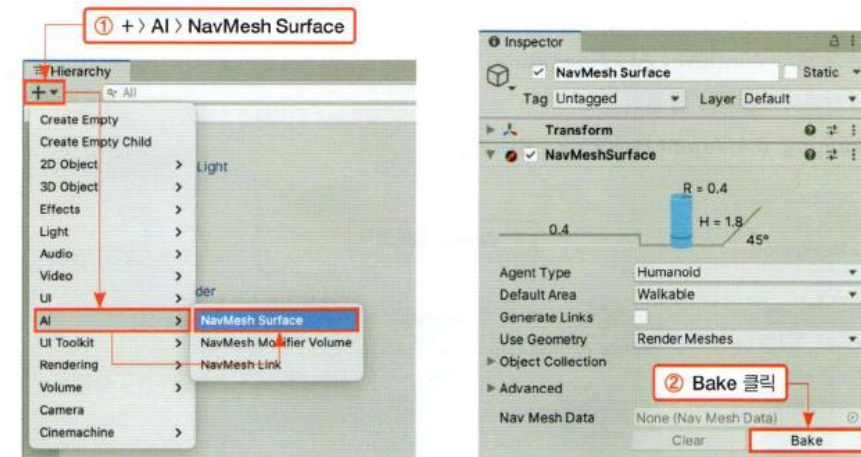
- ① 내비게이션 창 열기(Window > AI > Navigation)
- ② 내비게이션 창에서 Agents 탭 클릭
- ③ Agent Radius를 0.4, Agent Height를 1.8로 변경



▶ 에이전트 타입 설정 변경하기

[과정 02] 내비메시 굽기

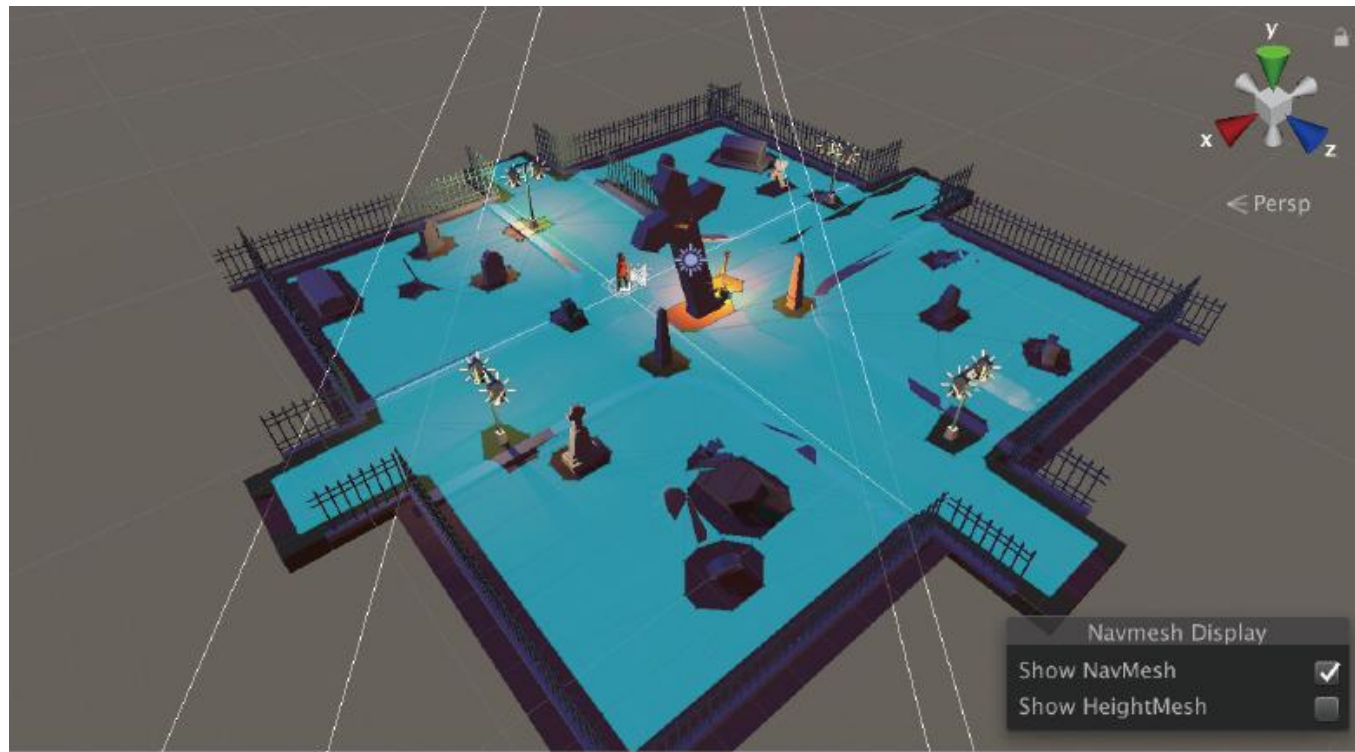
- ① 하이어라키 창에서 + > AI > NavMesh Surface
- ② NavMesh Surface 게임 오브젝트의 NavMeshSurface 컴포넌트의 Bake 버튼 클릭



▶ 내비메시 굽기

16.5 내비게이션 시스템과 좀비 준비(4)

- 구워진 내비메시는 씬 창에서 파란색으로 표시됩니다. 파란색으로 표시된 표면이 내비메시 에이전트가 이동 가능한 영역
- 과정 01에서 변경한 값은 구워진 에이전트의 키(Agent Height)와 반지름(Agent Radius)
- 구워진 에이전트는 씬을 돌아다니며 에이전트가 이동 가능한 영역을 측정



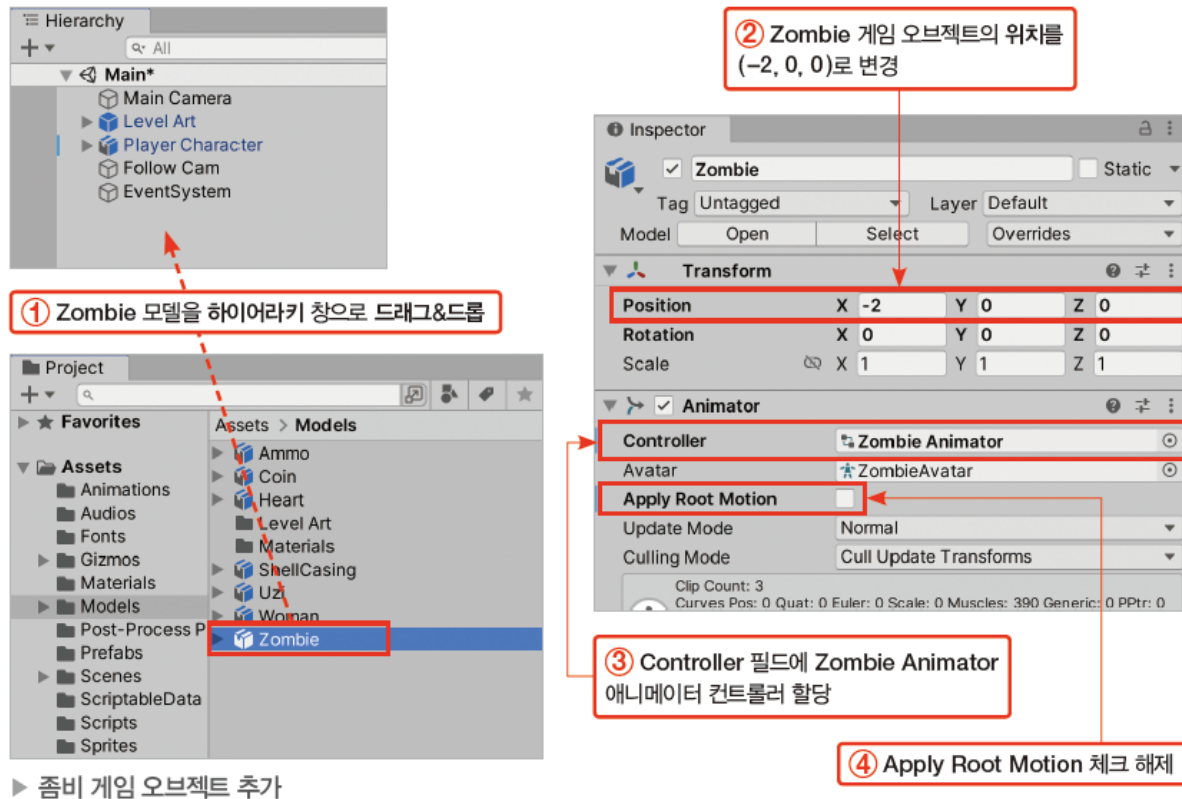
▶ 생성된 내비메시

16.5 내비게이션 시스템과 좀비 준비(5)

16.5.3 좀비 게임 오브젝트 준비

[과정 01] 좀비 게임 오브젝트 추가

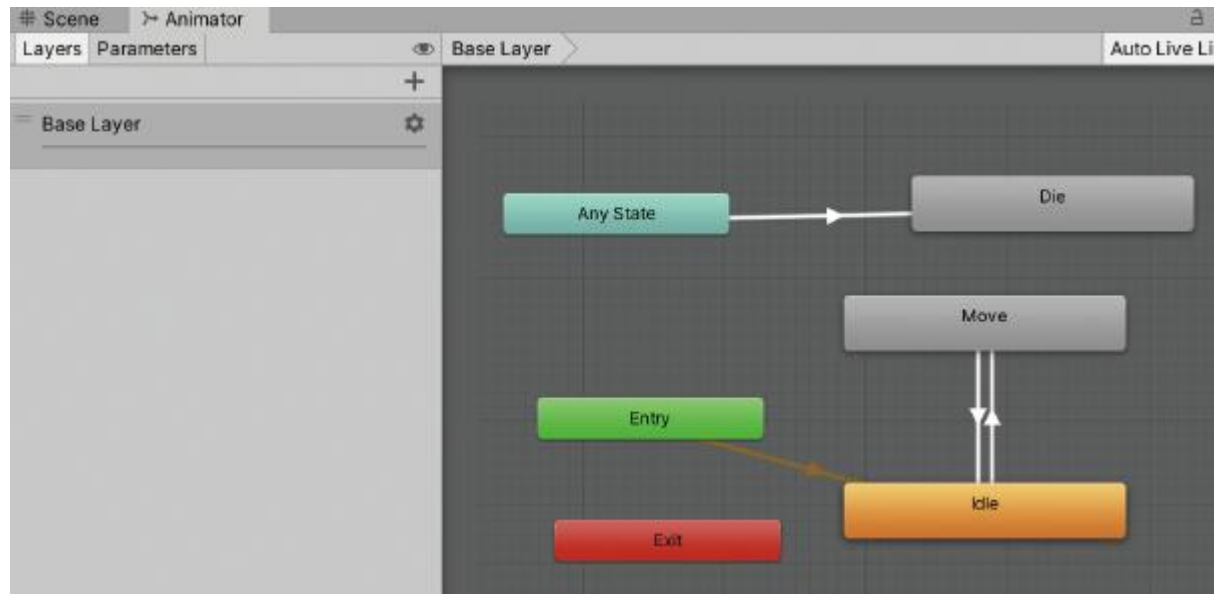
- ① Models 폴더에서 Zombie 모델을 하이어라키 창으로 드래그&드롭
- ② 생성된 Zombie 게임 오브젝트의 위치를 (-2, 0, 0)으로 변경
- ③ Animator 컴포넌트의 Controller 필드에 Zombie Animator 애니메이터 컨트롤러 할당(Controller 필드 옆 선택 버튼 클릭 > 선택 창에서 Zombie Animator 더블 클릭)
- ④ Animator 컴포넌트의 Apply Root Motion 체크 해제



16.5 내비게이션 시스템과 좀비 준비(6)

Zombie Animator 애니메이터 컨트롤러 구성

- Zombie Animator의 세 가지 상태
 - Idle : 가만히 서 있는 애니메이션 클립 재생
 - Move : 뛰는 애니메이션 클립 재생
 - Die : 사망 애니메이션 클립 재생
- 파라미터
 - Die : 트리거 타입. 사망 시 발동
 - HasTarget : bool 타입. 추적 대상이 있으면 true, 추적 대상이 없으면 false



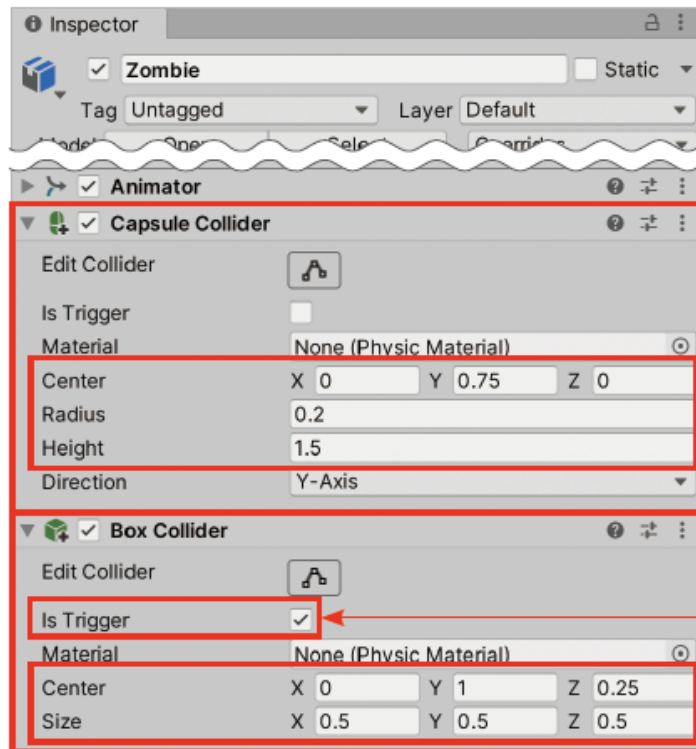
16.5 내비게이션 시스템과 좀비 준비(7)

16.5.4 좀비 컴포넌트 설정

- Zombie 게임 오브젝트에 필요한 컴포넌트를 추가

[과정 01] 콜라이더 추가

- ① Capsule Collider 컴포넌트 추가(Add Component > Physics > Capsule Collider)
- ② Capsule Collider 컴포넌트의 Center를 (0, 0.75, 0), Radius를 0.2, Height를 1.5로 변경
- ③ Box Collider 컴포넌트 추가(Add Component > Physics > Box Collider)
- ④ Box Collider 컴포넌트의 Is Trigger 체크
- ⑤ Box Collider 컴포넌트의 Center를 (0, 1, 0.25), Size를 (0.5, 0.5, 0.5)로 변경



① Capsule Collider 컴포넌트 추가

② Center를 (0, 0.75, 0), Radius를 0.2, Height를 1.5로 변경

③ Box Collider 컴포넌트 추가

④ Is Trigger 체크

⑤ Center를 (0, 1, 0.25), Size를 (0.5, 0.5, 0.5)로 변경

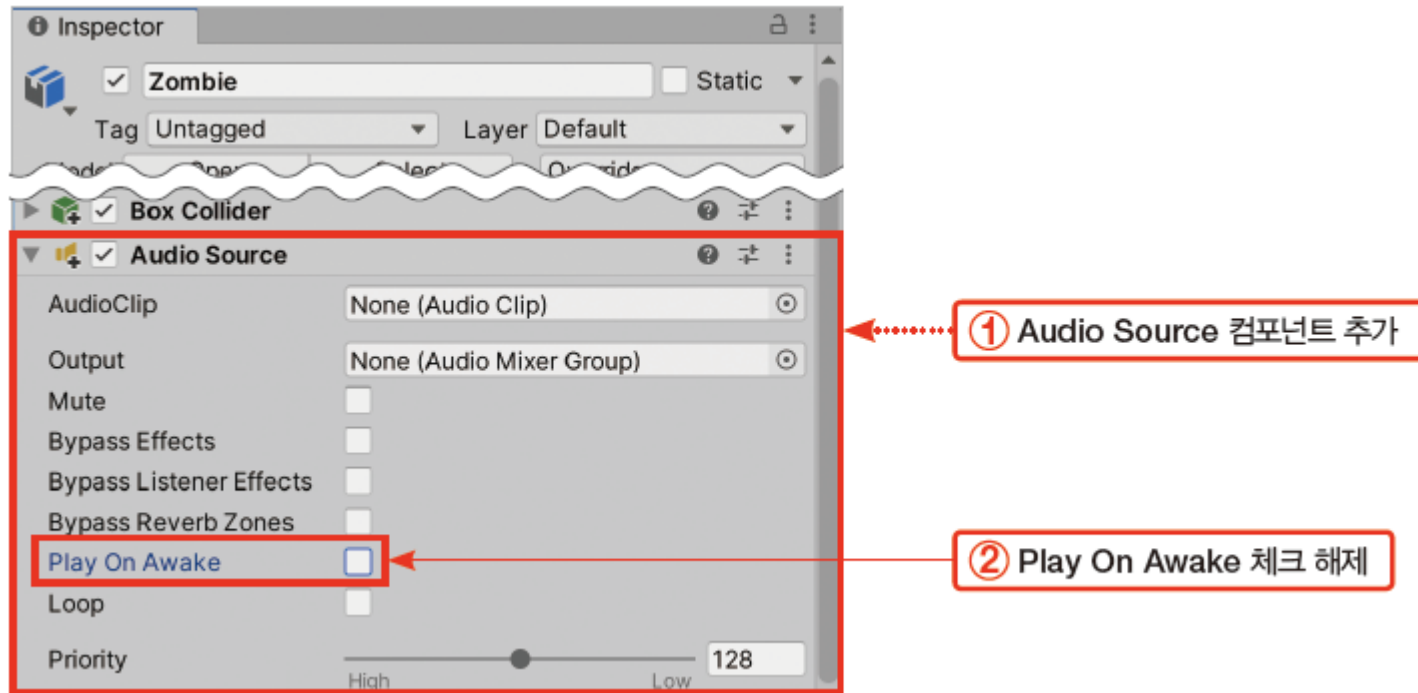
Box 콜라이더는 Trigger, 공격범위

16.5 내비게이션 시스템과 좀비 준비(8)

- 소리를 재생할 오디오 소스를 추가

[과정 02] 오디오 소스 추가하기

- ① Audio Source 컴포넌트 추가(Add Component > Audio > Audio Source)
- ② Audio Source 컴포넌트의 Play On Awake 체크 해제

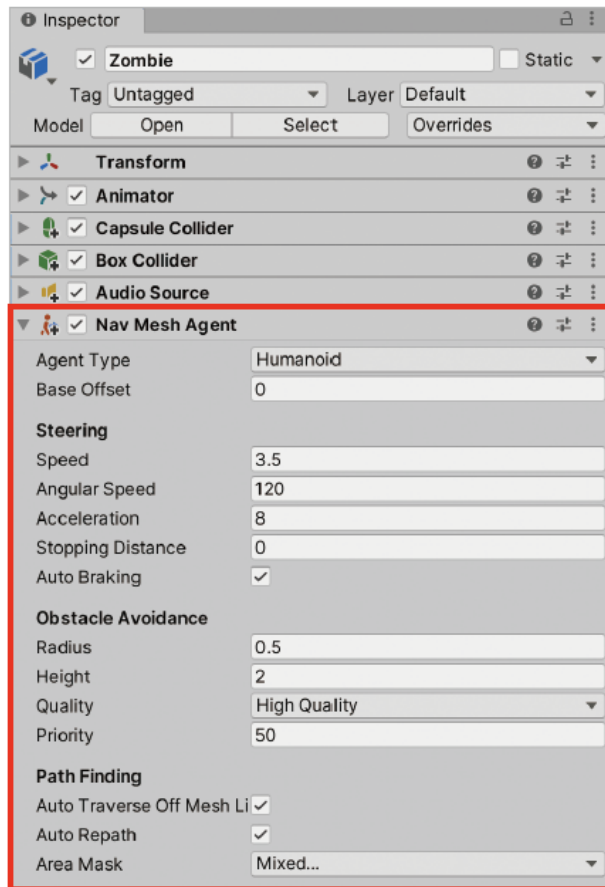


16.5 내비게이션 시스템과 좀비 준비(9)

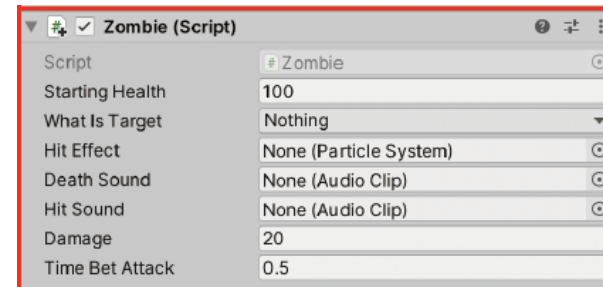
- Zombie 게임 오브젝트에 내비메시 에이전트 컴포넌트(경로 계산과 이동을 담당)와 Zombie 스크립트(내비메시 에이전트를 조종하여 좀비 AI를 구현)를 추가

[과정 03] 인공지능 추가하기

- ① Nav Mesh Agent 컴포넌트 추가(Add Component > Navigation > Nav Mesh Agent)
- ② Zombie 스크립트 추가(Scripts 폴더의 Zombie 스크립트를 Zombie 게임 오브젝트로 드래그&드롭)



① Nav Mesh Agent 컴포넌트 추가



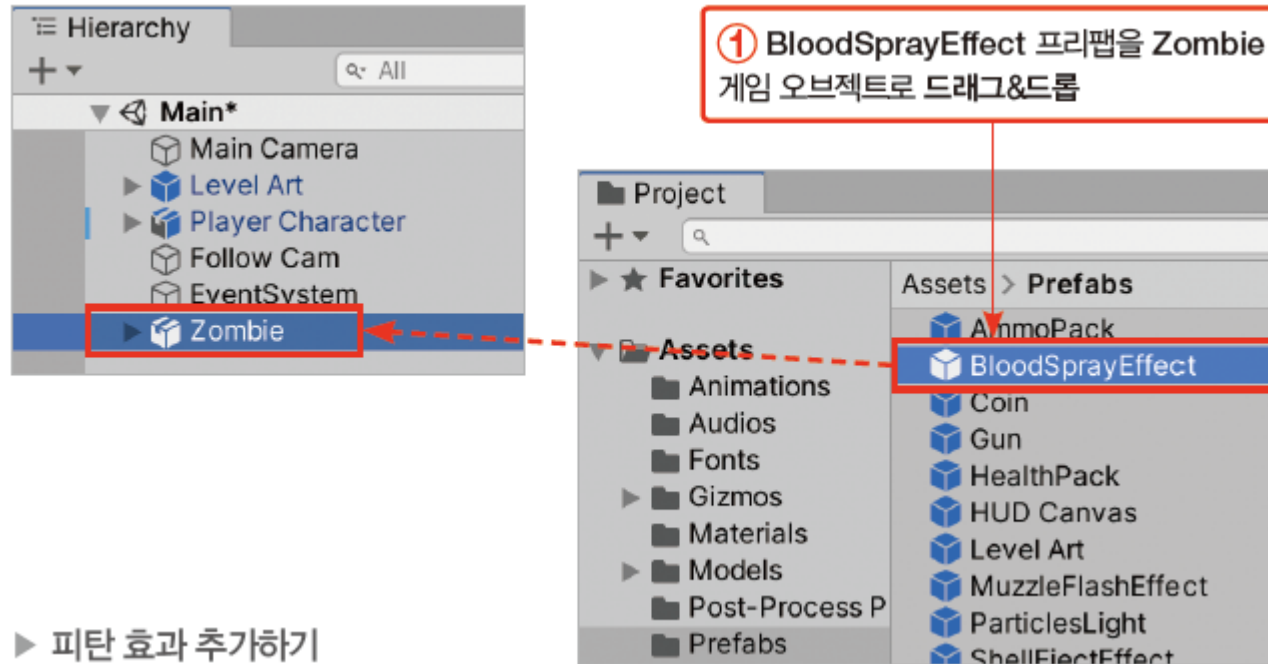
② Zombie 스크립트 추가

16.5 내비게이션 시스템과 좀비 준비(10)

- BloodSprayEffect 프리팹을 Zombie 게임 오브젝트의 자식으로 추가

[과정 04] 피탄 효과 추가하기

① Prefabs 폴더의 BloodSprayEffect 프리팹을 Zombie 게임 오브젝트로 드래그&드롭



16.6 Enemy 스크립트(1)

- Zombie 스크립트의 기능
 - LivingEntity에서 제공하는 기본 생명체 기능
 - 외부에서 Zombie 초기 능력치 셋업 가능
 - 주기적으로 목표 위치를 찾아 경로 갱신
 - 공격받았을 때 피탄 효과 재생
 - 트리거 콜라이더를 이용해 감지된 상대방을 공격
 - 사망 시 추적 중단
 - 사망 시 사망 효과 재생

16.6 Enemy 스크립트(2)

16.6.1 Enemy의 필드

- 씬에서 추적 대상을 검색할 때 사용할 레이어 마스크(LayerMask)가 선언

```
public LayerMask whatIsTarget;
```

- 추적할 대상을 할당할 변수 targetEntity가 LivingEntity 타입으로 선언

```
private LivingEntity targetEntity;
```

- NavMeshAgent 타입의 변수 navMeshAgent가 선언

```
private NavMeshAgent navMeshAgent;
```

- 피격 시 재생할 이펙트와 오디오에 대한 변수가 선언

```
public ParticleSystem hitEffect;
```

```
public AudioClip deathSound;
```

```
public AudioClip hitSound;
```

16.6 Enemy 스크립트(3)

- Zombie 게임 오브젝트의 컴포넌트에 대한 변수가 선언

```
private Animator zombieAnimator;  
private AudioSource zombieAudioPlayer;  
private Renderer zombieRenderer;
```

- 공격에 사용할 수치가 선언

```
public float damage = 20f;  
public float timeBetAttack = 0.5f;  
private float lastAttackTime;
```

- 추적할 대상이 존재하는지 알려주는 hasTarget 프로퍼티가 선언

```
private bool hasTarget  
{  
    get  
    {  
        if (targetEntity != null && !targetEntity.dead)  
        {
```

```
            return true;  
        }  
        return false;  
    }  
}
```

→ 위 코드 간단하게

16.6 Enemy 스크립트(4)

16.6.2 Awake()

- Enemy 클래스에서 가장 먼저 실행되는 Awake() 메서드는 사용할 컴포넌트를 찾아 변수에 할당

[과정 01] Awake () 메서드 완성하기

① Awake() 메서드를 다음과 같이 완성

```
private void Awake() {  
    // 게임 오브젝트로부터 사용할 컴포넌트 가져오기  
    navMeshAgent = GetComponent<NavMeshAgent>();  
    zombieAnimator = GetComponent<Animator>();  
    zombieAudioPlayer = GetComponent<AudioSource>();  
  
    // 렌더러 컴포넌트는 자식 게임 오브젝트에 있으므로  
    // GetComponentInChildren() 메서드 사용  
    zombieRenderer = GetComponentInChildren<Renderer>();  
}
```

16.6 Enemy 스크립트(5)

16.6.3 Setup()

- Setup() 메서드는 공격력, 체력, 이동 속도 등 적의 능력치를 설정
- Setup() 메서드는 public으로 지정되어 외부에서 사용할 수 있도록 공개

[과정 01] Setup() 메서드 완성하기

① Setup() 메서드를 다음과 같이 완성

```
public void Setup(ZombieData zombieData) {  
    // 체력 설정  
    startingHealth = zombieData.health;  
    health = zombieData.health;  
    // 공격력 설정  
    damage = zombieData.damage;  
    // 내비메시 에이전트의 이동 속도 설정  
    navMeshAgent.speed = zombieData.speed;  
    // 렌더러가 사용 중인 머티리얼의 컬러를 변경, 외형 색이 변함  
    zombieRenderer.material.color = zombieData.skinColor;  
}
```

17장에서 살펴볼 ZombieData 오브젝트의 필드

- float health : 체력
- float damage : 공격력
- float speed : 이동 속도
- Color skinColor : 피부색

16.6 Enemy 스크립트(6)

16.6.4 Start()와 Update()

- Start() 메서드는 경로 갱신을 위한 코루틴 UpdatePath()를 시작

```
private void Start() {  
    // 게임 오브젝트 활성화와 동시에 AI의 추적 루틴 시작  
    StartCoroutine(UpdatePath());  
}
```

- Update() 메서드는 애니메이터의 HasTarget 파라미터에 hasTarget 프로퍼티의 값을 할당하여 추적 대상의 존재 여부에 따라 알맞은 애니메이션을 재생

```
private void Update() {  
    // 추적 대상의 존재 여부에 따라 다른 애니메이션을 재생  
    zombieAnimator.SetBool("HasTarget", hasTarget);  
}
```

16.6 Enemy 스크립트(7)

16.6.5 UpdatePath()

- 추적할 대상의 갱신된 위치를 일정 주기로 파악하고, 인공지능의 목적지를 재설정하는 코루틴 메서드

[과정 01] UpdatePath () 메서드 완성하기

① UpdatePath() 메서드를 다음과 같이 완성

- while (!dead)로 AI 자신이 사망하지 않은 동안 처리를 영원히 반복하여 실행
- 만약 hasTarget이 true인 경우 추적 대상이 존재하므로 추적 및 이동을 계속 진행

```
if (hasTarget)
{
    navMeshAgent.isStopped = false;
    navMeshAgent.SetDestination(targetEntity.transform.position);
}
```

16.6 Enemy 스크립트(8)

- hasTarget이 false라면 추적할 대상이 없으므로 내비메시 에이전트 컴포넌트의 isStopped의 값을 true를 변경하여 추적과 이동을 중단

```
else
{
    navMeshAgent.isStopped = true;

    Collider[] colliders =
        Physics.OverlapSphere(transform.position, 20f, whatIsTarget);

    for (int i = 0; i < colliders.Length; i++)
    {
        LivingEntity livingEntity = colliders[i].GetComponent<LivingEntity>();

        if (livingEntity != null && !livingEntity.dead)
        {
            targetEntity = livingEntity;

            break;
        }
    }
}
```

16.6 Enemy 스크립트(9)

- 자신의 위치에서 화면에 보이지 않는 반지름 20유닛의 가상의 구를 그리고, 구와 겹치는 모든 콜라이더를 찾아 배열로 가져오기
 - Physics.OverlapSphere() 메서드는 다음을 배열로 가져옴
 - 자신의 위치 transform.position에서 반지름 20f 유닛의 가상의 구
 - 추적 대상인 레이어 마스크 whatIsTarget에 대응하는 레이어를 가진 콜라이더에 한해서
 - 구와 겹치는 모든 콜라이더

```
Collider[] colliders = Physics.OverlapSphere(transform.position, 20f, whatIsTarget);
```

16.6 Enemy 스크립트(10)

- 콜라이더 배열을 colliders로 가져온 다음 모든 콜라이더를 순회하면서 해당 콜라이더를 가진 게임 오브젝트가 살아 있는 LivingEntity인지 체크

```
// 모든 콜라이더를 순회하면서 살아 있는 LivingEntity 찾기
for (int i = 0; i < colliders.Length; i++)
{
    // 콜라이더로부터 LivingEntity 컴포넌트 가져오기
    LivingEntity livingEntity = colliders[i].GetComponent<LivingEntity>();

    // LivingEntity 컴포넌트가 존재하며 해당 LivingEntity가 살아 있다면
    if (livingEntity != null && !livingEntity.dead)
    {
        // 추적 대상을 해당 LivingEntity로 설정
        targetEntity = livingEntity;

        // for 문 루프 즉시 정지
        break;
    }
}
```

16.6 Enemy 스크립트(11)

- if 또는 else 블록의 모든 처리가 끝난 다음에는 마지막으로 while 문의 다음 루프 회차가 실행되기 전 yield 문을 사용하여 0.25초 동안 처리를 쉬
 - 즉, while 문 내부의 경로 갱신 코드는 좀비 AI가 살아 있는 동안 0.25초마다 반복 실행

```
yield return new WaitForSeconds(0.25f);
```

16.6 Enemy 스크립트(12)

◦ 16.6.6 OnDamage()

- OnDamage() 메서드는 공격받았을 때 실행할 처리를 구현
- Zombie 스크립트에서는 기존 OnDamage() 메서드에 파티클 효과와 효과음 재생을 추가

[과정 01] OnDamage () 메서드 완성

① OnDamage() 메서드를 다음과 같이 완성

- 피격 파티클 효과와 피격 효과음을 재생
 - 단 if (!dead)를 이용해 사망하지 않은 경우에만 파티클 효과와 효과음을 재생

```
if (!dead)
{
    hitEffect.transform.position = hitPoint;
    hitEffect.transform.rotation = Quaternion.LookRotation(hitNormal);
    hitEffect.Play();

    zombieAudioPlayer.PlayOneShot(hitSound);
}
```

16.6 Enemy 스크립트(13)

- 파티클 효과를 재생하기 전에 파티클 효과의 위치와 회전을 다음 값으로 변경
 - 위치 : 공격받은 지점(피격 위치)
 - 회전 : 공격이 날아온 방향을 바라보는 방향(피격 방향)

```
hitEffect.transform.position = hitPoint;  
hitEffect.transform.rotation = Quaternion.LookRotation(hitNormal);  
hitEffect.Play();
```

- 오디오 소스 컴포넌트를 사용해 hitSound에 할당된 피격 효과음을 재생

```
zombieAudioPlayer.PlayOneShot(hitSound);
```

- LivingEntity 클래스의 OnDamage() 메서드를 실행해 대미지를 실제로 적용

```
base.OnDamage(damage, hitPoint, hitNormal);
```

16.6 Enemy 스크립트(14)

16.6.7 Die()

[과정 01] Die () 메서드 완성

① Die() 메서드를 다음과 같이 완성

- base.Die();를 실행하여 기본 사망 처리를 실행
- 자신의 게임 오브젝트에 추가된 모든 콜라이더 컴포넌트를 찾아 비활성화

```
Collider[] zombieColliders = GetComponents<Collider>();  
for (int i = 0; i < zombieColliders.Length; i++)  
{  
    zombieColliders[i].enabled = false;  
}
```

- 사망한 좀비 AI가 더 이상 목표물을 향해 움직이지 않도록 내비메시 에이전트의 이동을 중단

```
navMeshAgent.isStopped = true;  
navMeshAgent.enabled = false;
```

- Die() 메서드 마지막 부분에서는 애니메이터와 오디오 소스를 사용해 사망 애니메이션과 사망 효과음을 재생

```
zombieAnimator.SetTrigger("Die");  
zombieAudioPlayer.PlayOneShot(deathSound);
```

16.6 Enemy 스크립트(15)

16.6.8 OnTriggerStay()

- 충돌한 상대방 게임 오브젝트가 자신이 공격하려는 대상이 맞는지 체크하고, 맞다면 상대방을 공격

[과정 01] Die () 메서드 완성

① Die() 메서드를 다음과 같이 완성

- if 문으로 현재 상태가 공격 가능한 상태인지 검사
 - !dead : 사망 상태가 아님
 - Time.time >= lastAttackTime + timeBetAttack : 최근 공격 시점에서 공격 주기 이상의 시간이 흐름

```
if (!dead && Time.time >= lastAttackTime + timeBetAttack)
```

- 공격 가능 조건을 만족하면 첫 if 문 블록이 실행
 - 첫 if 문 블록에서는 충돌한 상대방 게임 오브젝트에서 LivingEntity 컴포넌트를 가져와 attackTarget에 할당

```
LivingEntity attackTarget = other.GetComponent<LivingEntity>();
```

16.6 Enemy 스크립트(16)

- 두 번째 if 문에서는 attackTarget에 LivingEntity 타입이 할당되었는지, attackTarget이 자신의 추적 대상 targetEntity와 일치하는지 검사

```
if (attackTarget != null && attackTarget == targetEntity)
{
    lastAttackTime = Time.time;

    Vector3 hitPoint = other.ClosestPoint(transform.position);
    Vector3 hitNormal = transform.position - other.transform.position;

    attackTarget.OnDamage(damage, hitPoint, hitNormal);
}
```

- 상대방 콜라이더 표면에서 자신의 위치와 가장 가까운 점의 위치를 찾아 hitPoint로 사용

```
Vector3 hitPoint = other.ClosestPoint(transform.position);
```

- attackTarget의 OnDamage() 메서드를 실행하고 damage, hitPoint, hitTarget을 입력하여 공격을 실제로 실행

```
attackTarget.OnDamage(damage, hitPoint, hitNormal);
```

16.6 Enemy 스크립트(17)

16.6.9 완성된 Enemy 스크립트

```
using System.Collections;
using UnityEngine;
using UnityEngine.AI; // AI, 내비게이션 시스템 관련 코드 가져오기

// 좀비 AI 구현
public class Zombie : LivingEntity {
    public LayerMask whatIsTarget; // 추적 대상 레이어

    private LivingEntity targetEntity; // 추적 대상
    private NavMeshAgent navMeshAgent; // 경로 계산 AI 에이전트

    public ParticleSystem hitEffect; // 피격 시 재생할 파티클 효과
    public AudioClip deathSound; // 사망 시 재생할 소리
    public AudioClip hitSound; // 피격 시 재생할 소리

    private Animator zombieAnimator; // 애니메이터 컴포넌트
    private AudioSource zombieAudioPlayer; // 오디오 소스 컴포넌트
    private Renderer zombieRenderer; // 렌더러 컴포넌트
```

▶ 다음 쪽에 이어짐

16.6 Enemy 스크립트(18)

◀ 앞쪽에 이어

```
public float damage = 20f; // 공격력
public float timeBetAttack = 0.5f; // 공격 간격
private float lastAttackTime; // 마지막 공격 시점

// 추적할 대상이 존재하는지 알려주는 프로퍼티
private bool hasTarget
{
    get
    {
        // 추적할 대상이 존재하고, 대상이 사망하지 않았다면 true
        if (targetEntity != null && !targetEntity.dead)
        {
            return true;
        }

        // 그렇지 않다면 false
        return false;
    }
}

private void Awake() {
```

▶ 다음 쪽에 이어짐

16.6 Enemy 스크립트(19)

◀ 앞쪽에 이어

```
// 게임 오브젝트에서 사용할 컴포넌트 가져오기
navMeshAgent = GetComponent<NavMeshAgent>();
zombieAnimator = GetComponent<Animator>();
zombieAudioPlayer = GetComponent<AudioSource>();

// 렌더러 컴포넌트는 자식 게임 오브젝트에 있으므로
// GetComponentInChildren() 메서드 사용
zombieRenderer = GetComponentInChildren<Renderer>();
}
```

```
// 좀비 AI의 초기 스펙을 결정하는 셋업 메서드
public void Setup(ZombieData zombieData) {
    // 체력 설정
    startingHealth = zombieData.health;
    health = zombieData.damage;
    // 공격력 설정
    damage = zombieData.damage;
    // 내비메시 에이전트의 이동 속도 설정
    navMeshAgent.speed = zombieData.speed;
    // 렌더러가 사용 중인 머티리얼의 컬러를 변경, 외형 색이 변함
    zombieRenderer.material.color = zombieData.skinColor;
}
```

▶ 다음 쪽에 이어짐

16.6 Enemy 스크립트(20)

◀ 앞쪽에 이어

```
private void Start() {  
    // 게임 오브젝트 활성화와 동시에 AI의 추적 루틴 시작  
    StartCoroutine(UpdatePath());  
}
```

```
private void Update() {  
    // 추적 대상의 존재 여부에 따라 다른 애니메이션 재생  
    zombieAnimator.SetBool("HasTarget", hasTarget);  
}
```

// 추적할 대상의 위치를 주기적으로 찾아 경로 갱신

```
private IEnumerator UpdatePath() {  
    // 살아 있는 동안 무한 루프  
    while (!dead)  
    {  
        if (hasTarget)  
        {
```

▶ 다음 쪽에 이어짐

16.6 Enemy 스크립트(21)

◀ 앞쪽에 이어

```
// 추적 대상 존재 : 경로를 갱신하고 AI 이동을 계속 진행
navMeshAgent.isStopped = false;
navMeshAgent.SetDestination(
    targetEntity.transform.position);
}
else
{
    // 추적 대상 없음 : AI 이동 중지
    navMeshAgent.isStopped = true;

    // 20유닛의 반지름을 가진 가상의 구를 그렸을 때 구와 겹치는 모든 콜라이더를 가져옴
    // 단, whatIsTarget 레이어를 가진 콜라이더만 가져오도록 필터링
    Collider[] colliders =
        Physics.OverlapSphere(transform.position, 20f, whatIsTarget);

    // 모든 콜라이더를 순회하면서 살아 있는 LivingEntity 찾기
    for (int i = 0; i < colliders.Length; i++)
    {
```

▶ 다음 쪽에 이어짐

16.6 Enemy 스크립트(22)

◀ 앞쪽에 이어

```
// 콜라이더로부터 LivingEntity 컴포넌트 가져오기
LivingEntity livingEntity = colliders[i].GetComponent<LivingEntity>();

// LivingEntity 컴포넌트가 존재하며, 해당 LivingEntity가 살아 있다면
if (livingEntity != null && !livingEntity.dead)
{
    // 추적 대상을 해당 LivingEntity로 설정
    targetEntity = livingEntity;

    // for 문 루프 즉시 정지
    break;
}
}

// 0.25초 주기로 처리 반복
yield return new WaitForSeconds(0.25f);
}
}
```

▶ 다음 쪽에 이어짐

16.6 Enemy 스크립트(23)

◀ 앞쪽에 이어

```
// 대미지를 입었을 때 실행할 처리
public override void OnDamage(float damage,
    Vector3 hitPoint, Vector3 hitNormal) {
    // 아직 사망하지 않은 경우에만 피격 효과 재생
    if (!dead)
    {
        // 공격받은 지점과 방향으로 파티클 효과 재생
        hitEffect.transform.position = hitPoint;
        hitEffect.transform.rotation =
            Quaternion.LookRotation(hitNormal);
        hitEffect.Play();

        // 피격 효과음 재생
        zombieAudioPlayer.PlayOneShot(hitSound);
    }

    // LivingEntity의 OnDamage()를 실행하여 대미지 적용
    base.OnDamage(damage, hitPoint, hitNormal);
}
```

▶ 다음 쪽에 이어짐

16.6 Enemy 스크립트(24)

◀ 앞쪽에 이어

// 사망 처리

```
public override void Die() {  
    // LivingEntity의 Die()를 실행하여 기본 사망 처리 실행  
    base.Die();  
  
    // 다른 AI를 방해하지 않도록 자신의 모든 콜라이더를 비활성화  
    Collider[] zombieColliders = GetComponents<Collider>();  
    for (int i = 0; i < zombieColliders.Length; i++)  
    {  
        zombieColliders[i].enabled = false;  
    }  
  
    // AI 추적을 중지하고 내비메시 컴포넌트를 비활성화  
    navMeshAgent.isStopped = true;  
    navMeshAgent.enabled = false;  
  
    // 사망 애니메이션 재생  
    zombieAnimator.SetTrigger("Die");  
    // 사망 효과음 재생  
    zombieAudioPlayer.PlayOneShot(deathSound);  
}
```

▶ 다음 쪽에 이어짐

16.6 Enemy 스크립트(25)

◀ 앞쪽에 이어

```
private void OnTriggerStay(Collider other) {  
    // 자신이 사망하지 않았으며,  
    // 최근 공격 시점에서 timeBetAttack 이상 시간이 지났다면 공격 가능  
    if (!dead && Time.time >= lastAttackTime + timeBetAttack)  
    {  
        // 상대방의 LivingEntity 타입 가져오기 시도  
        LivingEntity attackTarget =  
            other.GetComponent<LivingEntity>();  
  
        // 상대방의 LivingEntity가 자신의 추적 대상이라면 공격 실행  
        if (attackTarget != null && attackTarget == targetEntity)  
        {  
            // 최근 공격 시간 갱신  
            lastAttackTime = Time.time;  
        }  
    }  
}
```

▶ 다음 쪽에 이어짐

16.6 Enemy 스크립트(26)

◀ 앞쪽에 이어

```
// 최근 공격 시간 갱신
lastAttackTime = Time.time;

// 상대방의 피격 위치와 피격 방향을 근사값으로 계산
Vector3 hitPoint =
    other.ClosestPoint(transform.position);
Vector3 hitNormal =
    transform.position - other.transform.position;

// 공격 실행
attackTarget.OnDamage(damage, hitPoint, hitNormal);
    }
}
}
```

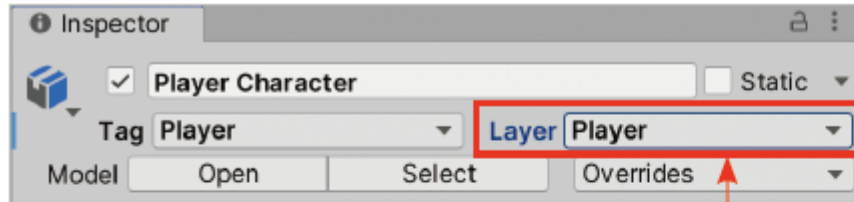
16.6 Enemy 스크립트(27)

16.6.10 Enemy 컴포넌트 설정

- 완성한 Enemy 컴포넌트의 필드들을 설정하고 좀비를 완성

[과정 01] Player Character에 레이어 할당

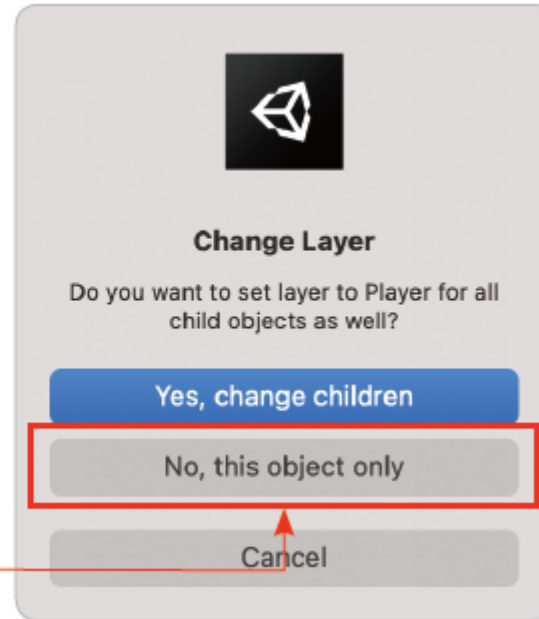
- ① Player Character 게임 오브젝트의 레이어를 Player로 변경(Layer 드롭다운 버튼 클릭 > Player 선택)
- ② 팝업 창에서 No, this object only 클릭



▶ Player Character에 레이어 할당

① 레이어를 Player로 변경(Layer 드롭다운 버튼 클릭 > Player 선택)

② 팝업 창에서 No, this object only 클릭

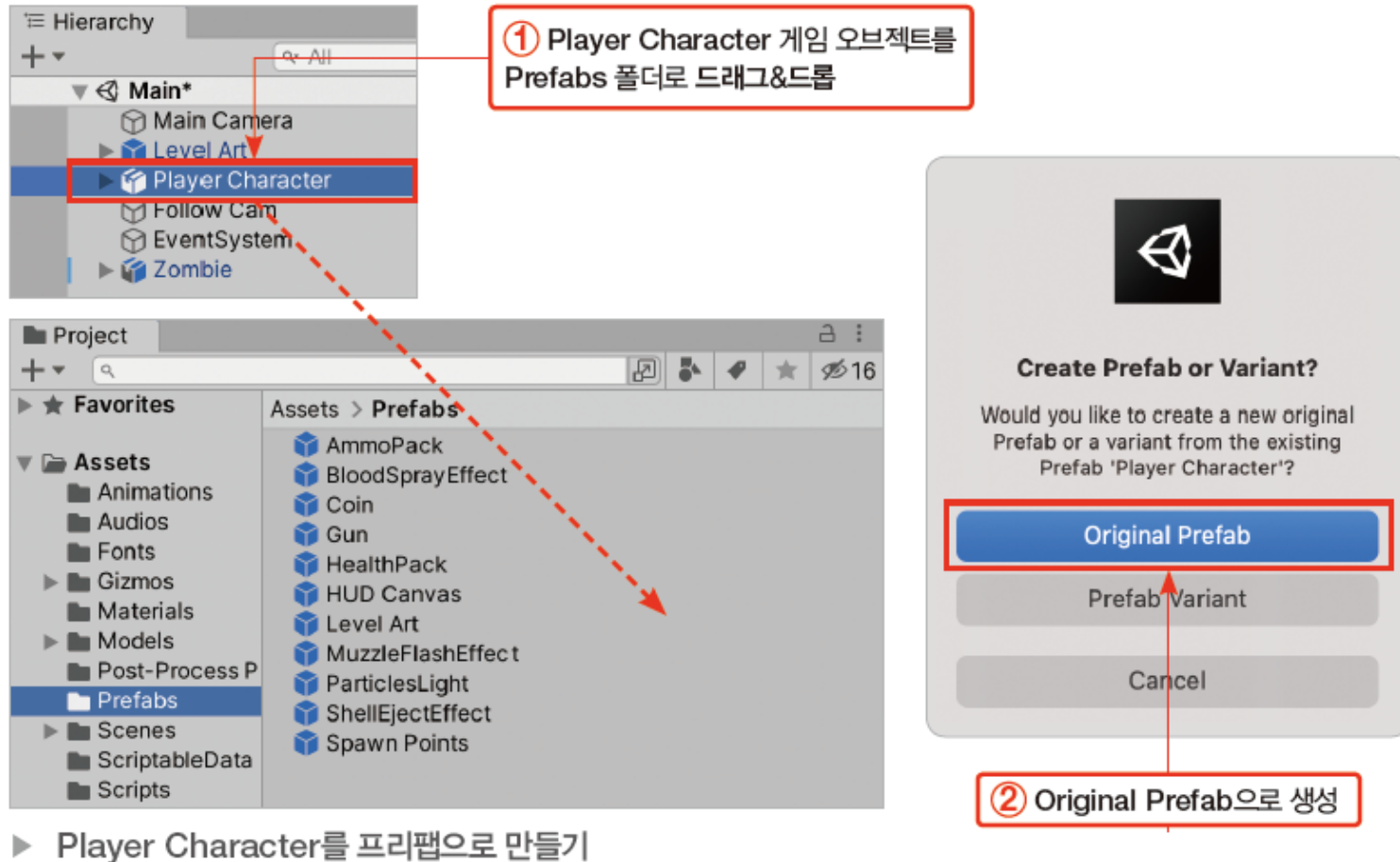


16.6 Enemy 스크립트(28)

- Player Character 게임 오브젝트를 별개의 Prefab으로 만들기

[과정 02] Player Chracter를 프리팹으로 만들기

- ① Player Character 게임 오브젝트를 하이어라키 창에서 프로젝트 창의 Prefabs 폴더로 드래그&드롭
- ② 팝업 창에서 Original Prefab으로 생성 선택



16.6 Enemy 스크립트(29)

[과정 03] Zombie 컴포넌트 설정

- ① What Is Target 레이어 마스크에 Player 레이어 등록(Nothing으로 표시된 드롭다운 버튼 클릭 > Player 선택)
- ② Hit Effect 필드에 Zombie의 자식으로 있는 BloodSprayEffect 게임 오브젝트 할당
- ③ Death Sound 필드에 Zombie Die 오디오 클립 할당
- ④ Hit Sound 필드에 Zombie Damage 오디오 클립 할당

The image shows the Unity Inspector and Hierarchy panels. The Inspector panel displays the 'Zombie' script with various fields. The 'What Is Target' field is highlighted with a red box, and a red arrow points to a dropdown menu with 'Player' selected. The 'Hit Effect' field is highlighted with a blue box, and a blue arrow points to the 'BloodSprayEffect (Particle System)' in the Hierarchy panel. The 'Death Sound' field is highlighted with a yellow box, and a yellow arrow points to the 'Zombie Die' audio clip in the 'Select AudioClip' dialog. The 'Hit Sound' field is highlighted with a blue box, and a blue arrow points to the 'Zombie Damage' audio clip in the 'Select AudioClip' dialog. The Hierarchy panel shows the 'Zombie' object with its children: 'Zombie_Cylinder' and 'BloodSprayEffect'. The 'Select AudioClip' dialog shows a list of audio clips, with 'Zombie Damage' and 'Zombie Die' highlighted.

① 드롭다운 버튼 클릭 > Player 선택

16.6 Enemy 스크립트(30)

- Zombie 컴포넌트 설정 완료

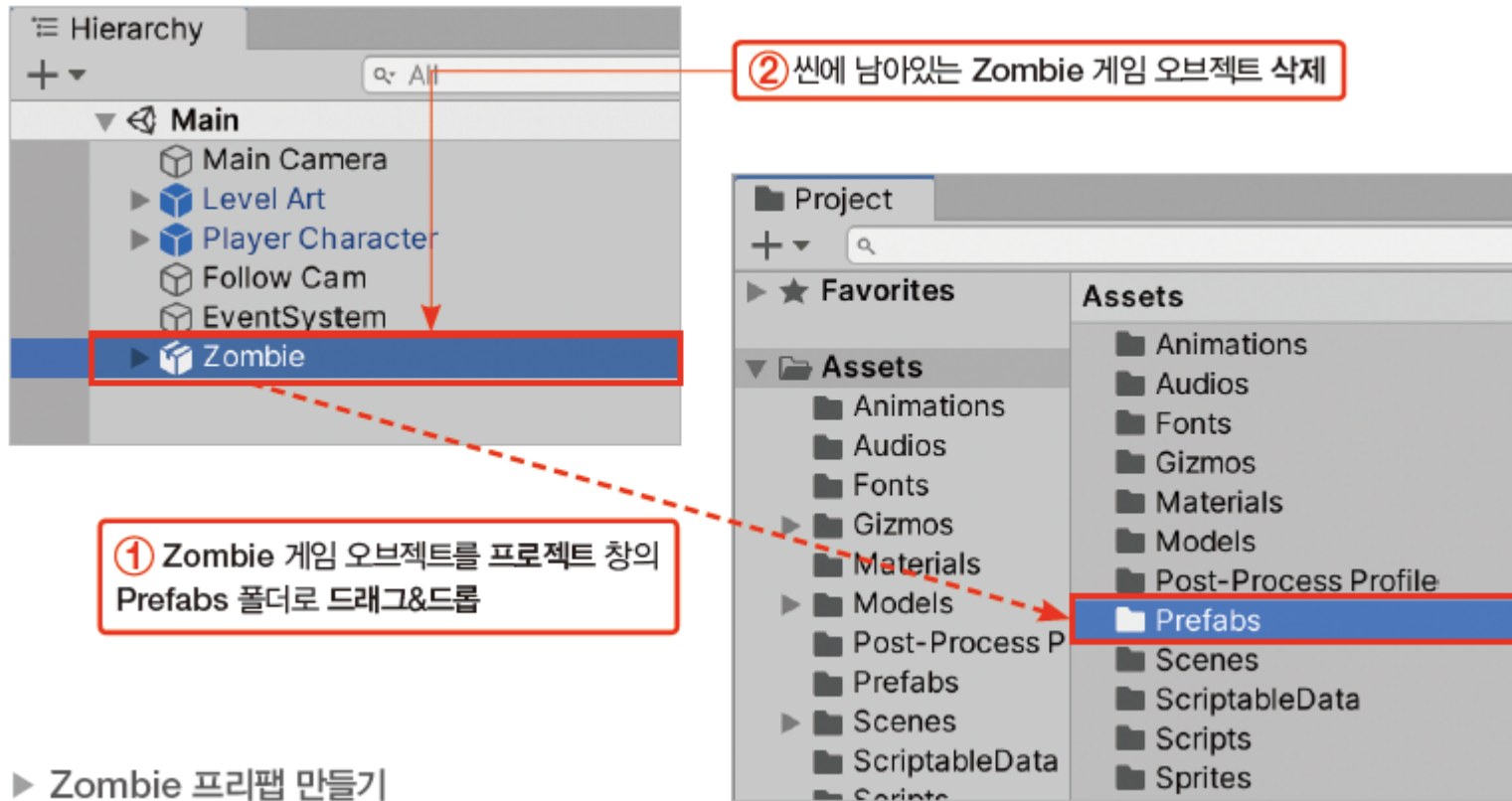


16.6 Enemy 스크립트(31)

- 완성된 Zombie 게임 오브젝트를 프리팹으로 만들기

[과정 04] Zombie 프리팹 만들기

- ① 하이어라키 창의 Zombie 게임 오브젝트를 프로젝트 창의 Prefabs 폴더로 드래그&드롭
- ② 씬에 남아 있는 Zombie 게임 오브젝트를 하이어라키 창에서 [Delete]로 삭제



16.7 마치며

이 장에서 배운 내용 요약

- 다형성을 사용하여 자식 클래스 타입의 오브젝트를 부모 클래스 타입으로 다룰 수 있음
- virtual로 선언된 가상 메서드는 자식 클래스에서 override로 재정의할 수 있음
- 델리게이트(delegate)는 메서드를 할당받아 원하는 시점에 할당된 메서드를 매번 실행할 수 있는 타입
- Action은 입력과 출력이 없는 메서드를 등록할 수 있는 델리게이트 타입
- 이벤트는 연쇄적인 처리를 발생시키는 사건
- 이벤트를 사용해 한 클래스에서 일어난 사건을 다른 클래스에서 알 수 있음
- 이벤트를 구독하는 메서드를 이벤트 리스너라고 부름
- 이벤트는 클래스 외부로 델리게이트 변수를 공개하여 구현
- event 키워드는 이벤트를 클래스 외부에서 발동할 수 없게 함
- 슬라이더 컴포넌트는 Value 값에 따라 슬라이더를 채움
- 슬라이더 컴포넌트는 슬라이더의 모습을 그리지 않음
- Fill 게임 오브젝트의 이미지 컴포넌트가 슬라이더의 모습을 그림
- UI 컴포넌트의 Interactable 필드를 체크 해제하면 상호작용이 불가능
- 캔버스 컴포넌트의 렌더 모드를 World Space로 변경하면 UI를 게임 월드에 배치할 수 있음
- 내비메시는 내비메시 에이전트가 이동할 수 있는 영역
- 내비메시는 실시간으로 생성되지 않으므로 미리 구워야 함
- 내비메시 에이전트는 내비메시 상의 한 점에서 다른 점으로 경로를 자동으로 계산하고 이동하는 인공지능 컴포넌트
- 내비메시 에이전트 컴포넌트의 SetDestination() 메서드를 사용해 에이전트가 이동할 위치를 지정할 수 있음
- 내비메시 에이전트 컴포넌트의 isStopped 필드를 사용해 에이전트의 이동을 멈추거나 재개